

Conception d'une plateforme de réservation de terrain de padel intégrant le Model Context Protocol

TRAVAIL DE BACHELOR HES RÉALISÉ EN VUE DE
L'OBTENTION DU BACHELOR PAR :

JONATHAN BOREL-JAQUET

CONSEILLERS AU TRAVAIL DE BACHELOR :
LUDOVIC BERTIN, ENSEIGNANT À LA HEG

GENÈVE, LE 22 JUIN 2026

Haute école de Gestion de Genève (HEG-GE)

Filière Informatique de gestion

1 Déclaration

Ce travail de Bachelor est réalisé dans le cadre de l'examen final de la Haute école de gestion de Genève, en vue de l'obtention du titre de Bachelor of Science HES-SO en Informatique de gestion.

L'étudiant-e atteste avoir réalisé seul-e le présent travail, sans avoir utilisé des sources autres que celles citées dans la bibliographie. Il ou elle atteste par ailleurs que le travail rendu est le fruit de sa réflexion personnelle et a été rédigé de manière autonome.

L'étudiant-e a envoyé ce document par email à l'adresse remise par son directeur de mémoire afin qu'il l'analyse à l'aide du logiciel de détection de plagiat COMPILATIO.

L'étudiant-e accepte, le cas échéant, la clause de confidentialité. L'utilisation des conclusions et recommandations formulées dans le travail de Bachelor, sans préjuger de leur valeur, n'engage ni la responsabilité de l'auteur, ni celle du ou de la conseiller-ère au travail de Bachelor, celle du juré-e ou celle de la HEG.

"J'atteste avoir réalisé seul le présent travail, sans avoir utilisé des sources autres que celles citées dans la bibliographie."

Fait à Genève, le 16 juin 2026

Jonathan Borel-Jaquet

2 Remerciements

Je tiens à remercier M. Hauri, professeur à la HEG, pour son aide précieuse dans le choix du sujet traité lors de ce travail de Bachelor.

Mes remerciements vont également à David Paulino, ancien étudiant de la HEG, pour son template LaTeX respectant les normes de l'école, qui m'a permis de gagner un temps précieux lors de la rédaction de ce document.

Je remercie tout particulièrement M. Bertin, professeur à la HEG et conseiller de ce travail de Bachelor, pour sa disponibilité et la pertinence de ses réponses tout au long de ce projet.

Enfin, je remercie mes proches, qui ont pris le temps de relire ce rapport et de me conseiller dans sa formulation.

3 Résumé

Liste des tableaux

1	Priorisation détaillée des livrables selon la méthodologie MoS-CoW	21
2	Planification Gantt du projet (Partie 1)	23
3	Planification Gantt du projet (Partie 2)	24
4	Dictionnaire de données - Table User	39
5	Dictionnaire de données - Table Club	40
6	Dictionnaire de données - Table Court	41
7	Dictionnaire de données - Table Match	42
8	Dictionnaire de données - Table Participant	43
9	Dictionnaire de données - Table LlmUsageLog	43
10	Paramètres météorologiques MeteoSwiss choisis pour le projet PadelContext (MeteoSwiss, 2026) [10]	45
11	Structure type des fichiers CSV de données de prévision locales de MeteoSwiss	45
12	Points de terminaison de l'API - Partie 1	47
13	Points de terminaison de l'API - Partie 2	48
14	Liste des outils exposés par le serveur MCP de PadelContext .	51

Table des figures

1	Aperçu de l'application PadelConnect	15
2	Aperçu de l'application PlayPad	16
3	Architecture du Model Context Protocol, adaptée de (Silva et al. 2025) [16]	18
4	Diagramme de cas d'utilisation de l'application PadelContext .	25
5	Diagramme de l'architecture logicielle de l'écosystème PadelContext	28
6	Mécanisme de transport local via Stdio, adapté de (Dohnal 2025) [20]	33
7	Mécanisme de transport réseau via Streamable HTTP, adapté de (Dohnal 2025) [20]	34
8	Schéma relationnel de la base de données de l'application PadelContext	38

Table des matières

1	Déclaration	1
2	Remerciements	2
3	Résumé	3
	Liste des tableaux	4
	Table des figures	5
4	Introduction	8
4.1	Introduction à la problématique	8
4.2	Intérêt de la problématique	9
4.3	Questions auxquelles nous souhaitons répondre dans ce travail	11
4.4	Plan du document	12
5	État de l’art	13
5.1	Solutions de réservation actuelles	13
5.1.1	PadelConnect	13
5.1.2	PlayPad	15
5.1.3	Limites des solutions existantes	16
5.2	Model Context Protocol	17
6	Méthodologie	19
6.1	MoSCoW	19
6.2	Gantt	22
7	Analyse et conception	24
7.1	Modélisation fonctionnelle	25
7.1.1	Diagramme de cas d’utilisation	25
7.1.2	Description des acteurs et des fonctionnalités	25
7.2	Architecture logicielle et écosystème	27
7.3	API REST et base de données	29
7.3.1	Choix de l’architecture de l’API	29
7.3.2	Choix du système de base de données	29
7.3.3	Choix du framework et des outils de développement . .	31
7.4	Serveur MCP	32

7.5	Client MCP et modèle de langage	35
7.6	Conteneurisation et gestion des environnements	36
7.7	Modélisation de la base de données	37
7.7.1	Schéma relationnel	37
7.7.2	Dictionnaire de données	38
7.8	Identification des sources de données météorologiques	44
8	Réalisation	46
8.1	API REST	46
8.1.1	Points de terminaison	46
8.1.2	Acquisition des données météorologiques	48
8.1.3	Stratégie de tests	49
8.2	Serveur MCP	50
8.2.1	Définitions des outils MCP	50
8.3	Client MCP et assistant IA	53
8.4	Fiabilité de l'assistant	53
8.5	Maîtrise des coûts	53
8.6	Cas d'usage	53
9	Évaluation des résultats	53
9.1	Bénéfice UX	53
9.2	Fiabilité face aux hallucinations	53
9.3	Viabilité des coûts et complexité	53
10	Conclusion	53
10.1	Conclusion générale	53
10.2	Améliorations possibles	53
	Références	53
11	Annexes	57
11.1	Diagramme gantt du projet PadelContext	57
11.2	Documentation Swagger de l'API PadelContext	58

4 Introduction

4.1 Introduction à la problématique

Le padel est un sport de raquette dérivé du tennis qui se joue exclusivement en double¹, sur un court de dimensions réduites et encadré de vitres ou de grillages. Au cours de ces dernières années, cette discipline a connu une croissance fulgurante à l'échelle mondiale, et plus particulièrement en Europe ainsi qu'en Amérique latine. Face à cet engouement massif, de nombreuses infrastructures ont été construites pour répondre à une demande toujours plus grandissante. Cependant, bien que l'accès à ce sport se démocratise, l'organisation et la réservation de ces terrains représentent encore aujourd'hui un véritable défi pour ses pratiquants.

Dans le cadre de ce travail de bachelor, nous allons nous intéresser plus précisément à la gestion et aux problématiques de réservation des terrains de padel dans le canton de Genève. Actuellement, l'offre genevoise se répartit sur plusieurs sites dont les plus connus sont les suivants :

- Centre sportif de Vessy : 1 terrain extérieur
- Centre sportif universitaire (CSU) : 1 terrain extérieur
- Centre sportif de Bernex : 2 terrains extérieurs
- Parc des Evaux : 3 terrains extérieurs
- Padel Academy, Jonction : 2 terrains couverts
- Club international de tennis, Pregny-Chambésy : 2 terrains intérieurs

Comme nous pouvons le constater, beaucoup de terrains de padel à Genève se situent en extérieur. Cette caractéristique amène un premier défi de taille, la dépendance à la météorologie. En effet, les conditions climatiques sont un facteur déterminant dans la pratique de ce sport. La pluie rend le jeu difficile, voire impossible, car elle modifie la physique de la balle et altère ses rebonds sur le sol et les vitres. Le vent dévie les trajectoires, tandis que l'éblouissement causé par le soleil complique fortement la lecture des trajectoires aériennes, notamment lors des lobs (nécessitant souvent l'anticipation avec des lunettes ou des casquettes).

1. Deux joueurs contre deux joueurs

Bien que les terrains en intérieur ou couvert permettent de s'affranchir de ces contraintes météorologiques, ils s'avèrent souvent plus coûteux, posant ainsi une barrière financière pour les joueurs disposant d'un budget limité.

Outre les facteurs environnementaux, la nature même du padel génère des contraintes organisationnelles complexes. Ce sport nécessitant impérativement la présence de quatre joueurs, la recherche de partenaires devient une problématique centrale. Il ne s'agit pas seulement de faire coïncider les disponibilités horaires de quatre individus, mais également de tenir compte du niveau de chacun afin de préserver l'équilibre et la qualité du jeu. Le système de réservation doit ainsi faciliter la mise en relation des joueurs. Il doit permettre à l'un de créer une partie ouverte en quête de partenaires, et aux autres de la rejoindre selon le niveau affiché et le nombre de places encore disponibles.

Enfin, l'aspect logistique ne s'arrête pas à la formation des équipes. De nombreux joueurs occasionnels ne possédant pas leur propre équipement, l'accès au matériel sur le lieu de pratique est primordial. La disponibilité de box de location (raquettes et balles) à proximité des terrains devient alors un critère de choix décisif lors de la réservation.

Au vu de la multitude de paramètres à prendre en compte, qu'il s'agisse des contraintes météorologiques, de l'homogénéité des niveaux, de la flexibilité du matchmaking, du budget ou de la disponibilité du matériel, l'organisation d'une partie s'apparente aujourd'hui à un véritable casse-tête logistique. Le traitement simultané de toutes ces variables dépasse largement les capacités d'un système de réservation traditionnel. La problématique de ce travail de bachelor est donc de concevoir PadelContext, une application de réservation centralisée et intelligente utilisant un modèle de langage couplé au Model Context Protocol, spécifiquement adaptée aux contraintes du padel genevois pour réduire ces difficultés.

4.2 Intérêt de la problématique

Aujourd'hui, la réservation de terrains de padel à Genève est un véritable casse-tête organisationnel. En effet, la multitude de variables à prendre en compte rend la planification d'une partie particulièrement complexe.

Parmi ces contraintes, la météo joue un rôle déterminant. Tout pratiquant souhaitant réserver un terrain en extérieur doit systématiquement croiser les disponibilités avec des données météorologiques (via des applications comme MeteoSwiss, par exemple) pour s'assurer de conditions de jeu favorables. À cela s'ajoute la difficulté de réunir quatre partenaires de niveau équivalent, ce qui s'avère fastidieux lorsque l'on manque de contacts ou que les agendas de chacun diffèrent. Par ailleurs, pour les joueurs non équipés, il est impératif de vérifier en amont si le centre sportif propose du matériel de location.

Actuellement, des applications comme PadelConnect ou PlayPad n'offrent pas de solution globale face à ces contraintes. Elles se concentrent presque exclusivement sur la location de l'infrastructure, délaissant la charge mentale liée à l'organisation et au suivi logistique de la partie.

L'intérêt de cette problématique est donc multiple. Premièrement, elle répond à une frustration réelle et croissante au sein de la communauté. Face à l'essor rapide du padel, la demande pour un outil centralisé et intelligent devient très intéressant. Une application dédiée permettrait non seulement de simplifier le processus, mais aussi d'améliorer considérablement l'expérience utilisateur grâce à des fonctionnalités spécifiques à ce sport.

Deuxièmement, cette démarche offre l'opportunité de concevoir une solution logicielle innovante, potentiellement transposable à d'autres sports collectifs. L'application pourrait révolutionner la manière dont les sportifs s'organisent, en rendant la planification plus fluide, rapide et accessible.

Surtout, la principale contribution de ce projet réside dans l'intégration d'un modèle de langage couplé au Model Context Protocol. Contrairement aux applications classiques qui se limitent à afficher une grille horaire statique et laissent l'utilisateur gérer seul la météo, le coût et la recherche de partenaires, cette architecture vise à transformer le système en un assistant conversationnel capable de prendre en charge ces démarches. En abaissant ainsi les barrières organisationnelles, une telle solution pourrait rendre le padel plus accessible et contribuer à sa démocratisation dans la région.

L'intérêt de ce travail ne se limite cependant pas à la démonstration de ces bénéfices. Déléguer l'organisation d'une réservation à un modèle de langage soulève des limites qu'il convient d'examiner avec la même rigueur. Il y a

d’une part le risque d’hallucinations, susceptible de compromettre la fiabilité des actions, et d’autre part le coût d’exploitation lié à la consommation de ressources du modèle de langage. L’enjeu est donc d’évaluer, de façon équilibrée, dans quelle mesure l’apport d’un tel assistant justifie réellement ces contraintes, un questionnement qui structure les questions de recherche présentées ci-après.

4.3 Questions auxquelles nous souhaitons répondre dans ce travail

Le but de ce travail est de répondre à trois grandes questions.

1. Dans quelle mesure un assistant conversationnel couplé au Model Context Protocol améliore-t-il l’expérience de réservation par rapport à une interface de réservation classique ?
2. Comment fiabiliser les actions de cet assistant face aux limites et aux problématiques des modèles de langage ?
3. L’intégration d’un tel assistant est-elle viable à l’usage, au regard de son coût et de son développement ?

Le premier objectif vise à mesurer l’apport réel d’un assistant conversationnel sur l’expérience de réservation. Pour cela, l’application proposera deux modes d’accès aux mêmes fonctionnalités. Une interface de navigation classique, où l’utilisateur effectue lui-même chaque étape, et un assistant conversationnel couplé au MCP², qui dialogue en langage naturel et orchestre ces étapes à sa place. Il s’agira alors de comparer ces deux approches, notamment en termes de temps et d’efforts nécessaires pour organiser une partie, afin de déterminer dans quelle mesure l’assistant améliore concrètement l’expérience utilisateur.

Le deuxième objectif s’attaque à une limite des modèles de langage. Premièrement, leur tendance à produire des réponses erronées ou inventées, aussi appelées hallucinations. Deuxièmement, dans un contexte où l’assistant exécute des actions concrètes, comme créer une partie ou y inscrire un joueur, une erreur n’est pas acceptable. Il s’agira de comprendre comment rendre fiables ces actions, d’une part en exposant les fonctionnalités sous forme

2. Acronyme de Model Context Protocol

d'outils structurés et explicitement décrits via le MCP, qui encadrent strictement ce que le modèle de langage peut faire et avec quels paramètres, et d'autre part en soumettant les actions sensibles à une validation humaine avant leur exécution.

Enfin, le troisième objectif consiste à évaluer la viabilité d'une telle intégration à l'usage. Au-delà de ses bénéfices, recourir à un modèle de langage engendre un coût d'exploitation directement lié à la consommation de tokens, qu'il convient de mesurer et d'encadrer. À cela s'ajoute la complexité de développement et de maintenance causée par l'orchestration entre l'assistant, le serveur MCP et l'application. L'enjeu sera donc de déterminer si l'apport de l'assistant justifie ces coûts, et à quelles conditions une telle solution demeure soutenable.

4.4 Plan du document

Ce rapport s'organise en plusieurs parties qui suivent la progression du projet, de la définition du problème à l'évaluation de la solution.

Tout d'abord, un état de l'art examine les solutions de réservation existantes sur le marché genevois afin de contextualiser la potentielle plus-value du projet, puis présente le Model Context Protocol dans le but d'expliquer son fonctionnement et ses avantages.

La suite du document décrit la méthodologie de gestion de projet adoptée, à travers la priorisation MoSCoW et la planification Gantt.

Ce rapport parcourt ensuite les trois axes majeurs du travail. Le premier concerne l'analyse et la conception, détaillant la recherche et la justification des besoins ainsi que des choix technologiques du projet. Le deuxième concerne la réalisation, détaillant l'implémentation concrète de l'API REST, du serveur MCP, du client MCP et de son assistant conversationnel, ainsi que les mécanismes de fiabilité et de maîtrise des coûts. Le troisième concerne l'évaluation des résultats, confrontant la solution à ses trois questions de recherche, à savoir le bénéfice pour l'expérience utilisateur, la fiabilité face aux hallucinations et la viabilité au regard des coûts.

Pour finir, une conclusion synthétise les réponses apportées, les limites du

travail et les améliorations envisageables.

5 État de l’art

5.1 Solutions de réservation actuelles

5.1.1 PadelConnect

PadelConnect est une application disponible sur Android, iOS et sur le web, qui permet actuellement de réserver des terrains de padel à Genève, plus précisément sur les quatre sites suivants :

- Centre sportif de Bernex
 - 2 terrains extérieurs
 - Tous les jours de 07h30 à 22h30
 - 10 CHF par personne
- Parc des Evaux
 - 3 terrains extérieurs
 - Tous les jours de 07h30 à 22h30
 - 10 CHF par personne
- Padel Academy, Jonction
 - 2 terrains couverts
 - Tous les jours de 09h00 à 21h30
 - 10 CHF par personne
- Club international de tennis, Pregny-Chambésy
 - 2 terrains intérieurs
 - Tous les jours de 05h30 à 01h30
 - 17 CHF par personne

L’application ne stipule pas si le terrain désiré propose une box de location de matériel ou non, il est donc de la responsabilité de l’utilisateur de vérifier cette information avant de réserver un terrain.

Celle-ci propose une interface simple pour consulter les disponibilités des terrains et y effectuer des réservations pour des sessions de 1h30. Il est possible de réserver un terrain complet de manière privée pour y inviter des personnes petit à petit, ou de rejoindre une partie publique existante avec de parfaits inconnus. Chaque réservation fournit sa propre page de discussion

permettant aux joueurs de communiquer entre eux.

Concernant les données météorologiques, l'application ne propose aucune fonctionnalité pour anticiper les conditions avant de réserver un terrain en extérieur, cette vérification reste donc à la charge de l'utilisateur. Toutefois, depuis le 27 octobre 2025, PadelConnect permet à ses utilisateurs d'annuler une partie et de se faire rembourser jusqu'à 180 minutes avant son début. À noter que la somme est remboursée sur le porte-monnaie de l'utilisateur, et non sur sa carte bancaire. Cette fonctionnalité permet de réserver un terrain en extérieur sans avoir à se soucier de la météo, puisqu'il est possible d'annuler et d'être remboursé si les conditions ne sont pas favorables.

L'application permet également d'y renseigner son niveau de jeu, allant de 1.0 à 10.0 par palier de 0.25, ainsi que son poste (gauche, droite ou indifférent). Ce niveau, totalement arbitraire et peu représentatif du niveau réel d'un joueur, permet tout de même un semblant de matchmaking en sélectionnant des parties avec des joueurs de notre « estimation » de niveau.

Après une partie, l'application permet aussi d'en insérer les résultats. Cette dernière fonctionnalité n'étant pas obligatoire, aucunement visible par les autres utilisateurs et non prise en compte dans le niveau du joueur, elle est le plus souvent ignorée.

Pour finir, l'application propose une fonctionnalité de recherche de joueurs grâce à différents filtres comme le niveau, la ville, l'âge et le sexe.

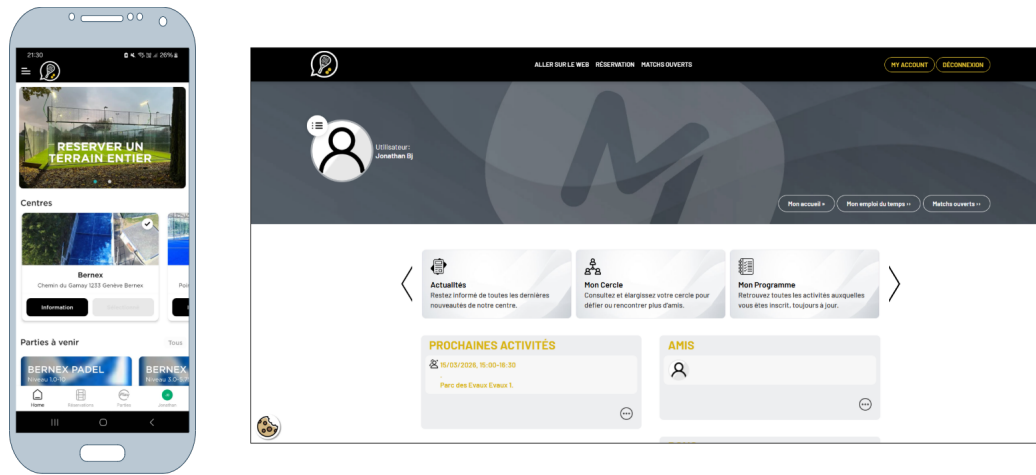


FIGURE 1 – Aperçu de l'application PadelConnect

5.1.2 PlayPad

PlayPad est une application récemment disponible sur Android et iOS. Actuellement, elle permet d'organiser des parties en Suisse romande ainsi qu'en France voisine. Bien qu'elle n'ait pas encore été lancée officiellement, elle intègre des fonctionnalités intéressantes.

Contrairement à PadelConnect, la particularité de PlayPad est de se concentrer essentiellement sur le classement des joueurs. Pour l'instant, l'application ne permet pas de réserver directement un terrain, mais seulement d'en choisir un parmi ceux disponibles, la réservation effective reste donc à la charge du créateur de la partie. À terme, l'application prévoit d'intégrer la réservation directe dans des clubs partenaires.

Comme mentionné précédemment, le cœur de l'application est son système de classement. En effet, les joueurs collectent des points au fil de leurs matchs pour grimper dans le classement général. Le niveau, toujours sur une base de 1 à 10, n'est plus une simple auto-évaluation. Il dépend des résultats obtenus sur le terrain. « Chaque victoire et défaite influence le niveau du joueur » (PlayPad 2024) [1]. L'application introduit également un score de fiabilité (de 0 à 100%) qui fonctionne comme un score d'assiduité. Plus le joueur est actif en mode compétitif, plus son pourcentage de fiabilité augmente.

On y retrouve un système de messagerie instantanée (via des demandes d'amis) ainsi qu'un chat dédié à chaque partie organisée.

Enfin, la saisie des résultats est publique, ce qui permet à l'ensemble de la communauté de suivre les performances de chacun.

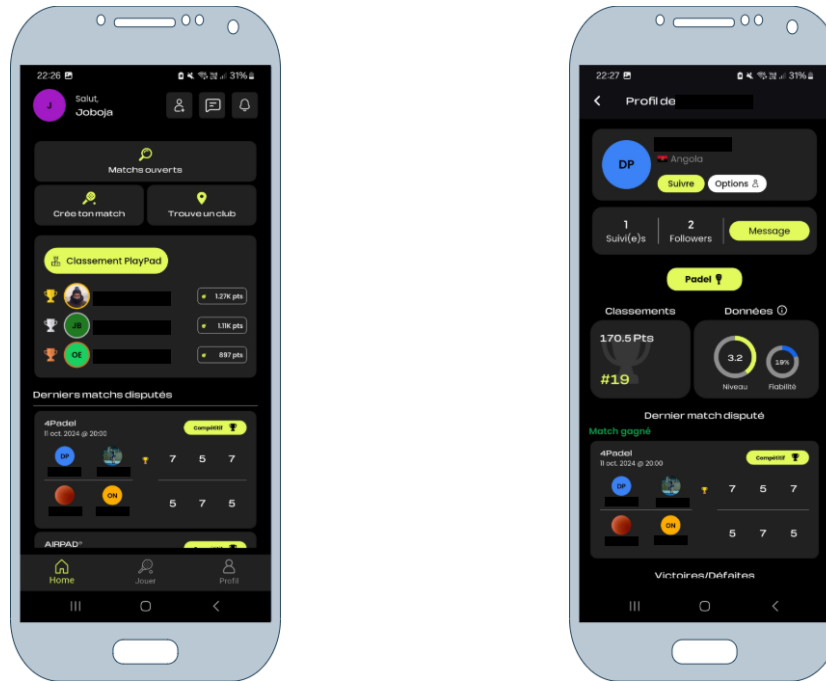


FIGURE 2 – Aperçu de l'application PlayPad

5.1.3 Limites des solutions existantes

L'analyse de ces deux solutions met en évidence un manque commun. Aucune ne décharge réellement l'utilisateur de la logistique d'organisation d'une partie. PadelConnect permet certes de réserver un terrain, mais laisse à l'utilisateur le soin de vérifier la météo, de s'assurer de la disponibilité du matériel et d'évaluer le niveau de ses partenaires. PlayPad, de son côté, se concentre sur le classement des joueurs et ne propose pas encore de réservation directe. Surtout, aucune de ces plateformes n'intègre d'assistant capable de croiser proactivement ces différentes contraintes.

5.2 Model Context Protocol

Le Model Context Protocol, abrégé MCP, est un standard ouvert dévoilé par Anthropic le 25 novembre 2024. Il est présenté comme un standard ouvert permettant d'établir des connexions bidirectionnelles sécurisées entre des sources de données et des outils alimentés par l'IA (Anthropic 2024) [4].

Ce protocole répond à une limite structurelle des modèles de langage. Même les plus avancés restent isolés des données, cloisonnés derrière des silos d'information, et chaque nouvelle source exige sa propre intégration sur mesure, ce qui empêche les systèmes de passer à l'échelle. Le MCP substitue alors à ces intégrations fragmentées un protocole unique et universel, capable de connecter un modèle de langage à des outils, des ressources et des prompts (Silva et al. 2025) [16].

Pour PadelContext, cela signifie qu'un modèle de langage n'a, par exemple, aucune connaissance du nombre de matchs enregistrés dans la base de données de l'application. Le MCP apporte précisément une couche permettant de décrire, de découvrir et d'invoquer de telles fonctions externes à l'exécution, sans qu'aucune intégration n'ait à être codée en dur dans le modèle de langage.

Sur le plan architectural, le MCP repose sur un modèle client-serveur. Les serveurs MCP exposent des interfaces fonctionnelles, des métadonnées et des mécanismes d'invocation, tandis que les clients MCP, par exemple des agents fondés sur un modèle de langage, interrogent dynamiquement ces serveurs pour découvrir les opérations disponibles et les exécuter au moyen de requêtes bien définies. Cette découverte à l'exécution constitue l'un des principaux atouts du protocole. Elle découple le modèle de langage des fonctions externes, de sorte que celui-ci n'a pas besoin d'avoir été entraîné avec la connaissance de ces outils et choisit, selon le contexte et l'objectif, la fonctionnalité à mobiliser.

L'architecture distingue par ailleurs deux types de serveurs. Les serveurs directs hébergent des capacités purement virtuelles ou calculatoires, exécutées localement sur l'hôte. Les serveurs passerelles, à l'inverse, jouent un rôle d'intermédiaire pour déclencher ou interroger des ressources externes, notamment des API ou des bases de données, en encapsulant la logique d'appel

derrière des interfaces telles que HTTP.

La figure 3 synthétise une architecture MCP. L'utilisateur interagit avec un client MCP, qui transmet au modèle de langage sa requête accompagnée des outils exposés par les serveurs MCP. Le modèle de langage renvoie sa réponse en désignant l'outil à employer. Le client invoque alors cet outil auprès du serveur MCP, selon le protocole MCP et au format JSON-RPC. L'hôte héberge les serveurs, qui mettent à disposition leurs capacités sous trois formes, à savoir des outils, des ressources et des prompts. Un serveur direct exécute l'opération localement tandis qu'un serveur passerelle relaie l'appel vers une ressource externe comme une API ou une base de données, via HTTP, avant d'en renvoyer le résultat.

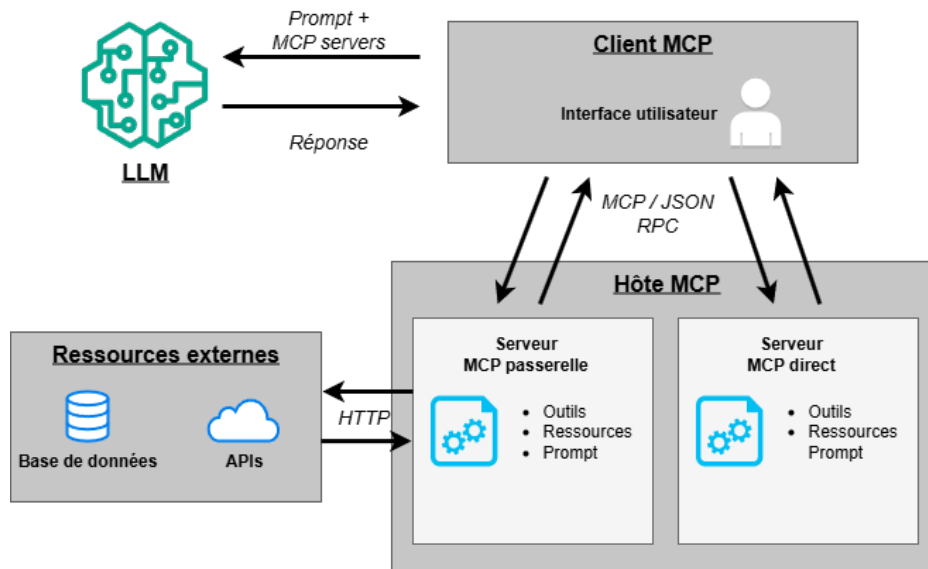


FIGURE 3 – Architecture du Model Context Protocol, adaptée de (Silva et al. 2025) [16]

Si le MCP simplifie considérablement l'intégration des modèles de langage à des systèmes externes, l'orchestration de tâches par un modèle de langage via ce protocole demeure une approche récente, dont la fiabilité et la robustesse, en particulier face aux hallucinations, restent un défi majeur.

6 Méthodologie

6.1 MoSCoW

Dans le cadre de ce travail de bachelor, je me suis appuyé sur la méthodologie MoSCoW afin de classer par ordre d'importance les différents livrables de mon projet. Cette démarche m'a permis de définir un périmètre clair et précis avant le début de la phase de développement du projet. Périmètre qui m'a permis de gagner en efficacité et de me concentrer sur les éléments les plus importants pour la réussite de mon projet, tout en gardant une certaine flexibilité pour intégrer des fonctionnalités supplémentaires si le temps et les ressources le permettent.

La méthode MoSCoW est une technique de priorisation qui a gagné en popularité en raison de sa grande simplicité et de sa facilité d'adoption. Elle permet aux parties prenantes de gérer les attentes et les compromis techniques, tout en se concentrant sur la livraison des éléments les plus importants en classant les fonctionnalités dans les quatre catégories suivantes :

- **Must-have (Indispensable)** : Les exigences qui comportent des spécifications vitales à la réussite du projet et qui doivent être satisfaites dans les délais impartis. Si ces éléments ne sont pas mis en œuvre, le projet échouera ou aura un impact négatif majeur sur les utilisateurs (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 2) [19].
- **Should-have (Souhaitable)** : Prérequis importants qui sont attrayants mais pas indispensables à la réussite immédiate du projet. Ils ajoutent de la valeur globale au produit et devraient être inclus une fois que toutes les fonctionnalités essentielles ont été satisfaites (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 3) [19].
- **Could-have (Possible)** : Critères supplémentaires qui améliorent le produit mais ne sont pas nécessaires à ses fonctionnalités principales. Ces éléments ne sont pas prioritaires par rapport aux éléments indispensables ou souhaitables, mais ils peuvent être réalisés si le temps et les ressources le permettent (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 3) [19].
- **Won't-have (Non souhaitable pour le moment)** : Besoins qui sont spécifiquement exclus du périmètre actuel du projet. Ils sont soit

considérés comme peu prioritaires pour le cycle de développement en cours, soit reportés à des versions ultérieures (trad. de Vijayakumar, Prasad K, Holla M 2024, p. 3) [19].

Dans l'objectif de la réalisation de la méthode MoSCoW, je me suis appuyé sur les recommandations du *DSDM Agile Project Framework Handbook* de l'Agile Business Consortium. Parmi ces recommandations, la plus déterminante concerne la répartition stratégique de l'effort de développement.

Afin de garantir la viabilité du projet et de se prémunir contre les imprévus, le référentiel préconise que les exigences classées comme indispensables (Must-have) ne doivent pas excéder 60 % de l'effort global alloué au projet. Les 40 % restants agissent comme une marge de contingence et sont alloués aux fonctionnalités souhaitables (Should-have) et possibles (Could-have), généralement réparties à parts égales soit 20 % chacune (Agile Business Consortium 2014) [5].

L'application stricte de cette règle permet d'assurer qu'en cas de complexité sous-estimée ou de contraintes de temps inattendues sur les éléments critiques, le travail de bachelor pourra toujours être livré dans les délais fixés, quitte à sacrifier temporairement des fonctionnalités de moindre priorité. C'est sur la base de ce cadre temporel et de ces pourcentages d'effort que le tableau suivant a été construit.

ID	Fonctionnalité / Tâche	Catégorie	Effort
01	Modélisation de la base de données et création de l'API de base	Must-have	15%
02	Mise en place du serveur MCP et intégration du modèle IA	Must-have	10%
03	Outil MCP (BDD) : Recherche filtrée des infrastructures (localisation basique par texte, prix, indoor/outdoor, box matériel)	Must-have	10%
04	Outil MCP (BDD) : Matchmaking (recherche de parties ouvertes selon le niveau et les places disponibles)	Must-have	10%
05	Outil MCP (API) : Interrogation des prévisions météorologiques locales pour la validation des terrains extérieurs	Must-have	10%
06	Outil MCP (BDD) : Exécution de la réservation (verrouillage du créneau et intégration des joueurs)	Must-have	5%
07	Création d'une application web minimaliste pour la gestion des réservations	Should-have	10%
08	Étude comparative entre les plateformes traditionnelles et la solution MCP	Should-have	10%
09	Application mobile native (iOS / Android)	Could-have	10%
10	Outil MCP (BDD) : Calculs avancés du niveau réel des joueurs par rapport à leur historique de parties	Could-have	5%
11	Outil MCP (Géolocalisation) : Filtrage avancé des terrains par coordonnées GPS	Could-have	5%
12	Interface de messagerie instantanée entre les joueurs	Won't-have	-
13	Païement intégré lors de la réservation d'un terrain	Won't-have	-

TABLE 1 – Priorisation détaillée des livrables selon la méthodologie MoSCoW

6.2 Gantt

Dans la continuité de la méthodologie MoSCoW, la réalisation d'une planification Gantt du projet m'a permis d'organiser et visualiser l'ensemble des tâches à réaliser, leur ordre chronologique et leur durée estimée. Elle a également permis d'identifier les dépendances entre les différentes tâches et d'anticiper les éventuels retards ou obstacles qui auraient pu survenir au cours du projet.

Les jalons, identifiés par un losange noir (◆), représentent les points clés du projet, tels que la mise en place d'une base de données et d'une API opérationnelles, ou encore la capacité du serveur MCP à interroger les données internes de l'application et l'API MeteoSwiss. Ces jalons sont essentiels pour mesurer l'avancement du projet et s'assurer que les objectifs intermédiaires sont atteints avant de passer à la phase suivante.

Le diagramme de Gantt est disponible dans l'annexe 11.1, cependant un aperçu est présenté ci-dessous.

Nom de la tâche	Date de début	Date de fin
Introduction de la documentation	02.03.26	06.03.26
Méthodologie et planification	09.03.26	13.03.26
Backend	16.03.26	10.04.26
Modélisation et documentation de la base de données	16.03.26	20.03.26
Étude, choix et documentation des technologies backend	23.03.26	27.03.26
Création et mise en place du serveur backend	30.03.26	03.04.26
Documentation de l'architecture backend	06.04.26	10.04.26
♦ Base de données et API opérationnelles	13.04.26	13.04.26
MCP - Base de données	13.04.26	24.04.26
Documentation et création des tools MCP avec les données internes	13.04.26	17.04.26
Création du MVP pour tester les tools	20.04.26	24.04.26
♦ Le serveur MCP est capable d'interroger les données internes de l'application	27.04.26	27.04.26
MCP - API MeteoSwiss	27.04.26	08.05.26
Documentation et création des tools MCP avec l'API MeteoSwiss	27.04.26	01.05.26
Création du MVP pour tester les tools	04.05.26	08.05.26
♦ Le serveur MCP est capable d'interroger l'API MeteoSwiss	11.05.26	11.05.26

TABLE 2 – Planification Gantt du projet (Partie 1)

Nom de la tâche	Date de début	Date de fin
Frontend	11.05.26	22.05.26
Documentation de l'architecture générale	11.05.26	15.05.26
Création de l'application WEB minimaliste	18.05.26	22.05.26
♦ Prototype complet et testable	25.05.26	25.05.26
Étude comparative	25.05.26	29.05.26
Rédaction de l'analyse comparative (PadelContext vs PadelConnect)	25.05.26	29.05.26
♦ Étude complétée et documentée	01.06.26	01.06.26
Fonctionnalités optionnelles et conclusion	01.06.26	19.06.26
Développement de l'application mobile native PadelContext	01.06.26	05.06.26
Documentation de la conclusion et des améliorations possibles	08.06.26	12.06.26
Finalisation de la documentation	15.06.26	19.06.26
♦ Rendu final du Travail de Bachelor	22.06.26	22.06.26

TABLE 3 – Planification Gantt du projet (Partie 2)

7 Analyse et conception

Ce chapitre établit les fondations du projet PadelContext en faisant le lien entre les besoins à couvrir et les décisions de conception qui en découlent. La réflexion débute par la modélisation fonctionnelle, qui formalise les acteurs du système et leurs cas d'usage, puis présente une vue d'ensemble de l'architecture logicielle et de son écosystème. Les sections suivantes détaillent et justifient les choix technologiques propres à chaque brique de la solution imaginée, de l'API REST au serveur MCP jusqu'au client MCP et au modèle de langage utilisé. Le chapitre se termine sur les aspects de conteneurisation, de modélisation de la base de données et de l'identification des sources de données météorologiques.

7.1 Modélisation fonctionnelle

7.1.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présenté à la figure 4 synthétise les interactions entre les utilisateurs et l'application PadelContext. Sa particularité tient à la place centrale de l'assistant conversationnel, qui s'intercale entre le joueur et les fonctionnalités du système et en devient un acteur à part entière.

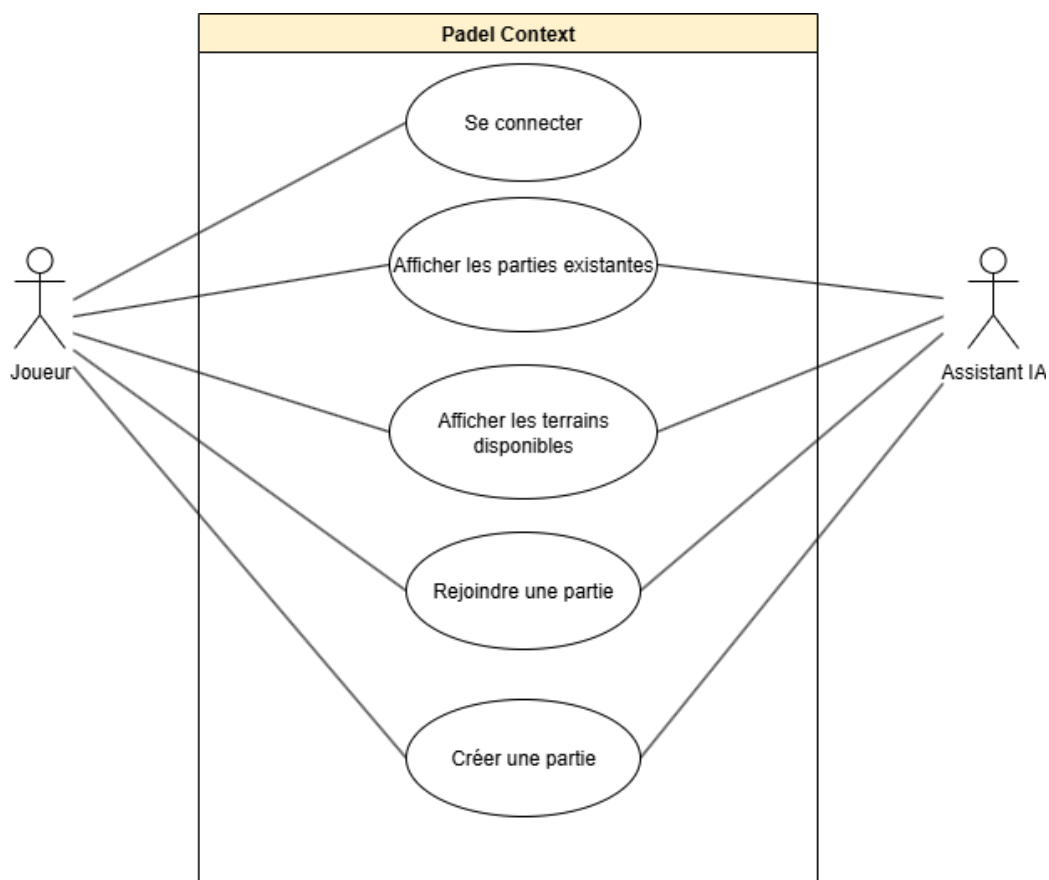


FIGURE 4 – Diagramme de cas d'utilisation de l'application PadelContext

7.1.2 Description des acteurs et des fonctionnalités

Afin de bien délimiter le périmètre d'action de l'application PadelContext, il est essentiel de définir les différents acteurs interagissant avec le système,

ainsi que les scénarios d'utilisation principaux qui en découlent. Contrairement à une application classique, l'intégration du Model Context Protocol modifie l'interaction entre les acteurs. L'utilisateur ne se renseigne pas et ne navigue pas uniquement via des clics dans des menus, mais dialogue avec un Assistant IA qui orchestre les requêtes de manière proactive.

Les acteurs principaux de l'application sont les suivants :

- **Le Joueur** : Il s'agit de l'utilisateur final de l'application. Son objectif est de trouver des partenaires, de consulter la disponibilité des terrains et de planifier des parties de padel tout en s'affranchissant de la charge mentale logistique (météo, équipement sur place, niveau des autres joueurs, etc.).
- **L'Assistant IA** : Utilisant un modèle de langage et connecté au serveur MCP, il agit comme un assistant proactif. Il comprend les requêtes en langage naturel du joueur et utilise les outils mis à sa disposition pour interroger l'API REST de l'application.

Les scénarios d'utilisation principaux sont les suivants :

- **Se connecter** : Le joueur s'authentifie sur l'application. Cette étape permet de personnaliser ses futures requêtes auprès de l'Assistant IA et de l'autoriser à exécuter des actions nécessitant une identification (comme rejoindre ou créer une partie).
- **Afficher les parties existantes** : Le joueur exprime à l'Assistant IA son désir de participer à une partie, par exemple en formulant « Je cherche à rejoindre une partie pour demain à 18h00 avec des joueurs d'un niveau moyen de 5.0 ». L'Assistant IA fait appel au serveur MCP pour interroger l'outil approprié. Il analyse les parties ouvertes et filtre les résultats pour ne proposer que ceux correspondant au niveau du joueur et à la tranche horaire souhaitée. Pour les parties se déroulant en extérieur, les prévisions météorologiques associées au créneau sont également présentées au joueur.
- **Rejoindre une partie** : Si le joueur valide la proposition de l'Assistant IA, ce dernier exécute l'action d'inscription à la partie sélectionnée, que le système réserve aux utilisateurs authentifiés.
- **Afficher les terrains disponibles** : Le joueur formule le souhait de réserver un terrain pour une date et une heure précises, par exemple

en formulant « Je souhaite réserver un terrain extérieur pour samedi prochain à 14h00 ». L'Assistant IA interroge le serveur MCP pour lister les terrains disponibles et filtre les résultats selon la tranche horaire et le type de terrain demandés. De plus, pour les terrains extérieurs, la proposition de l'Assistant IA intègre les prévisions météorologiques que le système associe à chaque créneau, afin que le joueur puisse juger des conditions de jeu.

- **Créer une partie** : Une fois la proposition validée par le joueur, l'Assistant IA crée une nouvelle partie ouverte sur le créneau choisi et y inscrit le joueur, une action que le système réserve aux utilisateurs authentifiés.

7.2 Architecture logicielle et écosystème

L'architecture de PadelContext est conçue comme un système distribué où chaque fonctionnalité est isolée dans son propre environnement d'exécution. Grâce à un réseau virtuel privé dédié, les différents conteneurs Docker qui composent l'écosystème de l'application communiquent entre eux de manière sécurisée et efficace.

Le diagramme de la figure 5 synthétise les différents services qui composent cet écosystème ainsi que leurs interactions.

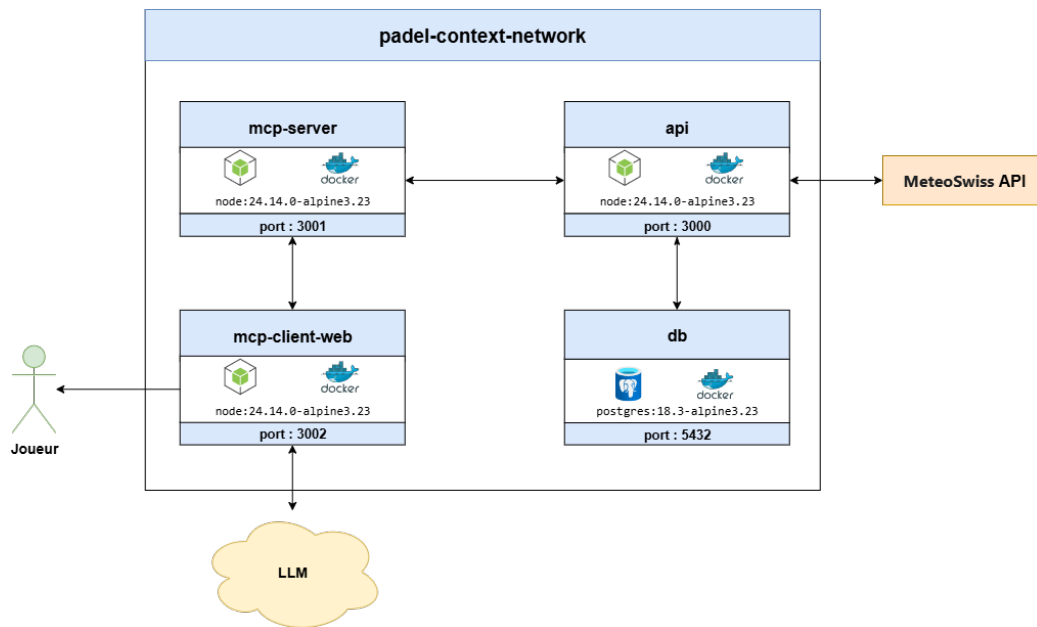


FIGURE 5 – Diagramme de l'architecture logicielle de l'écosystème PadelContext

L'écosystème repose sur quatre services conteneurisés. Le client MCP web constitue le point d'entrée du joueur et pilote le modèle de langage qui anime l'assistant conversationnel. Il transmet les requêtes au serveur MCP, lequel expose les fonctionnalités de l'application sous forme d'outils et relaie les appels vers l'API REST. Cette dernière centralise la logique métier et l'accès aux données. Elle interroge la base de données PostgreSQL pour les informations propres à l'application, et l'API externe MeteoSwiss pour enrichir les créneaux extérieurs de prévisions météorologiques.

Le choix de confier la récupération des données météorologiques à l'API, plutôt qu'à un outil dédié du serveur MCP, constitue un parti pris d'architecture détaillé dans la section consacrée à la maîtrise des coûts du projet.

Sur le plan de la configuration, les ports utilisés pour les communications internes entre les conteneurs sont fixés en dur dans le code de chaque service, tandis que les ports exposés sur la machine hôte restent configurables via des variables d'environnement. Cette approche conserve une structure interne stable tout en préservant la souplesse nécessaire au déploiement.

7.3 API REST et base de données

7.3.1 Choix de l'architecture de l'API

Dans l'objectif d'assurer l'échange de données entre les interfaces utilisateur et la base de données, tout en fournissant un point d'accès standardisé au serveur MCP, le développement d'une API s'est avéré indispensable.

Pour ce faire, il a d'abord fallu déterminer l'architecture d'API la plus adaptée aux contraintes du projet. Parmi les standards actuels les plus connus se trouvent les architectures SOAP, REST et GraphQL. Dans le cadre de ce projet, j'ai choisi d'utiliser l'architecture REST. En effet, contrairement à l'architecture SOAP qui repose sur des messages XML et une structure de communication plus rigide, l'architecture REST prône la simplicité et une communication sans état.

De plus, REST s'intègre parfaitement aux standards WEB en privilégiant fortement l'utilisation de formats de données légers tels que JSON, ce qui en fait aujourd'hui le standard pour la conception d'API web. Cette légèreté est idéale pour garantir des temps de réponse rapides entre l'API et ses clients, à savoir le serveur MCP et l'application web.

Enfin, bien que GraphQL émerge actuellement comme un leader pour la conception d'API dans les applications hautement dynamiques nécessitant des requêtes de données très complexes, son implémentation dans le cadre de ce travail de bachelor aurait ajouté une complexité superflue. Les opérations liées à une application de réservation de terrain de padel suivent une logique transactionnelle standard pour laquelle la simplicité éprouvée de l'architecture REST est amplement suffisante et parfaitement adaptée (Addanki 2025) [3].

7.3.2 Choix du système de base de données

Afin de stocker et gérer efficacement les données nécessaires à l'application et dans le but de permettre à l'API REST d'écrire et fournir des données, il a fallu choisir quel type de système de base de données utiliser entre une base de données relationnelle SQL et une base de données non relationnelle NoSQL.

Le choix le plus judicieux a été une base de données relationnelle SQL. En effet, les bases de données relationnelles sont particulièrement adaptées pour gérer des données structurées avec des relations complexes entre les différentes entités, ce qui est le cas dans une application de réservation de terrains de padel. Les données telles que les utilisateurs, les réservations, les matchs, les terrains, etc., sont toutes interconnectées et nécessitent une gestion rigoureuse des relations entre elles.

De plus, le respect des propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) inhérent aux bases de données relationnelles permet de garantir une grande fiabilité des données lors des transactions. Dans le contexte d'une plateforme de réservation, cela s'avère indispensable pour gérer les accès concurrents, comme s'assurer que deux groupes de joueurs ne puissent pas réserver le même terrain au même instant, ou garantir qu'une opération d'enregistrement s'annule complètement si une erreur survient en cours de processus.

« Dès lors que les propriétés ACID sont indispensables pour une base de données, il ne fait aucun doute que le seul choix possible est la base de données relationnelle. » (trad. de Sahatqija et al. 2018, p. 5) [14].

Une fois le choix du type de système de base de données effectué, il a fallu choisir quel SGBDR (Système de Gestion de Base de Données Relationnel) utiliser. Parmi les deux options open-source les plus populaires se trouvent MySQL et PostgreSQL.

Suite à l'analyse d'une étude récente de 2024 comparant les performances de ces deux moteurs de base de données, mon choix s'est définitivement porté sur PostgreSQL. En effet, cette recherche a mis en évidence la performance significative de PostgreSQL par rapport à MySQL à travers plusieurs tests simulant des conditions de production. Les résultats ont notamment démontré que le temps d'exécution pour la récupération d'une très grande quantité d'enregistrements était environ 13 fois plus rapide avec PostgreSQL qu'avec MySQL. Par ailleurs, pour les requêtes de sélection intégrant une clause de recherche spécifique, PostgreSQL s'est également révélé être environ 9 fois plus efficace que MySQL (Salunke et Ouda 2024, p. 1) [15].

7.3.3 Choix du framework et des outils de développement

Pour clôturer les choix technologiques liés à la conception du backend, il a fallu déterminer le framework à utiliser pour développer l'API REST. Mon choix s'est porté sur l'environnement Node.js associé au framework Express.js. Cette décision s'articule autour de trois critères majeurs.

Le premier critère concerne la popularité du langage JavaScript. En effet, selon l'enquête de 2025 de Stack Overflow, JavaScript demeure le langage de programmation le plus utilisé sur la plateforme (Stack Overflow 2025) [18]. Comme Stack Overflow est un forum largement répandu et reconnu, la consultation de celui-ci est avantageuse lors de la réalisation d'un projet académique de cette envergure.

Le second critère tient à la cohérence technologique de l'écosystème du projet. Le serveur MCP et le client MCP web reposant eux aussi sur Node.js et l'écosystème JavaScript, retenir Node.js pour l'API permet d'unifier le langage sur l'ensemble des services et de faciliter leur développement.

Le troisième critère repose sur l'expérience acquise. Avoir déjà utilisé Express.js lors d'un projet sur mandat durant mes études à la HEG m'a procuré une bonne maîtrise de ce framework et du langage JavaScript, ce qui me permet de développer l'API plus efficacement.

Afin de gagner en efficacité et de réduire le temps de développement de l'API REST, j'ai également choisi d'utiliser Prisma pour la gestion de la base de données. Cet ORM (Object-Relational Mapping) simplifie considérablement les interactions avec la base de données en offrant une interface de programmation intuitive. Il permet d'automatiser des tâches chronophages et critiques telles que la validation des données, la gestion des migrations structurelles, et la génération de requêtes SQL optimisées et sécurisées.

Au-delà du confort de développement, ce choix s'appuie sur des critères de performance concrets. Prisma offre les meilleures performances lorsqu'il est soumis à des charges parallèles par rapport à des alternatives comme Sequelize ou TypeORM (Zhadko-Bazilevych 2025) [21]. Pour une plateforme de réservation comme PadelContext, où plusieurs utilisateurs peuvent solliciter l'API simultanément pour rechercher ou réserver un créneau via le serveur MCP, cette capacité à maintenir de hautes performances sous une charge parallèle est un atout technique décisif.

Enfin, pour assurer la fiabilité de l'API REST, j'ai opté pour le framework de test Jest. Jest est un framework qui ne nécessite aucune configuration au préalable. De plus, il fournit un système de mocks intégré ainsi qu'une couverture de code complète (Donvir 2024) [6]. De manière similaire à Prisma, le choix de Jest est principalement justifié par sa rapidité d'intégration et son efficacité, dans l'objectif d'optimiser le temps alloué au développement de l'API REST.

7.4 Serveur MCP

Comme présenté dans l'état de l'art, le rôle d'un serveur MCP est d'exposer des fonctionnalités à un modèle de langage à travers une interface standardisée par le protocole. Dans le cas de PadelContext, le serveur expose des outils, désignés par le terme *tools* dans la terminologie MCP, qui permettent au modèle de langage de consulter des données et d'exécuter des actions. Chaque outil définit une opération précise, dotée d'entrées et de sorties typées, que le modèle déclenche en fonction du contexte de la conversation.

Pour faciliter son implémentation, le Model Context Protocol fournit des kits de développement logiciel (SDK) pour de nombreux langages, parmi lesquels PHP, Python, Java ou encore TypeScript. C'est ce dernier qui a été retenu pour le projet. Ce SDK fournit l'ensemble des abstractions et des composants logiciels nécessaires à la création d'un serveur MCP, en masquant la complexité de l'implémentation du protocole. C'est sur cette base que repose le développement du serveur MCP de PadelContext.

Le protocole MCP propose nativement deux mécanismes de transport pour acheminer les messages entre le client et le serveur. Le premier, nommé *Stdio*, repose sur le lancement du serveur en tant que sous-processus du client. La communication s'effectue alors localement, via les flux d'entrée et de sortie standards du système d'exploitation (Model Context Protocol 2025) [11], comme l'illustre la figure 6.

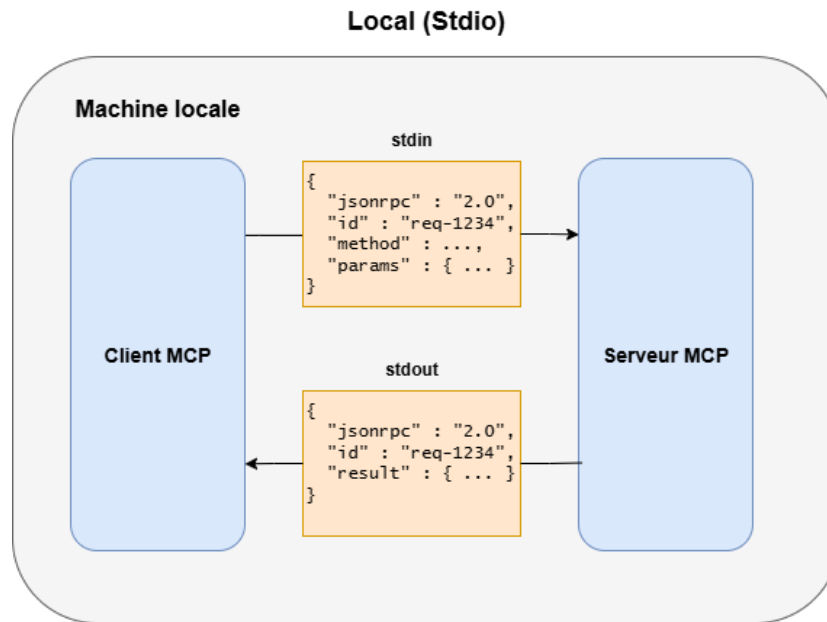


FIGURE 6 – Mécanisme de transport local via Stdio, adapté de (Dohnal 2025) [20]

À l'inverse, le mécanisme Streamable HTTP permet au serveur de fonctionner comme un processus réseau indépendant. Il s'appuie sur des requêtes HTTP POST pour l'envoi des commandes et sur des requêtes HTTP GET pouvant établir un flux continu, comme le montre la figure 7.

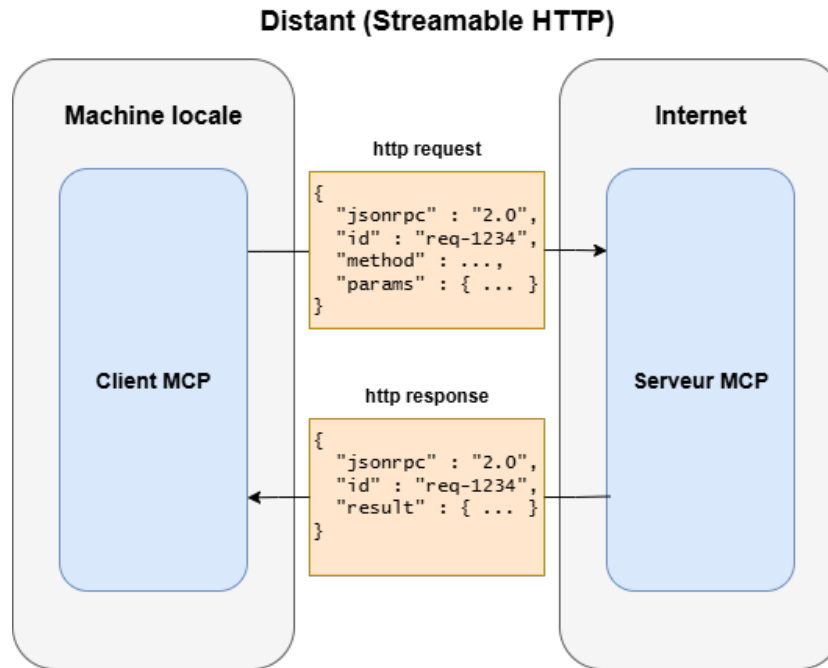


FIGURE 7 – Mécanisme de transport réseau via Streamable HTTP, adapté de (Dohnal 2025) [20]

Pour PadelContext, le choix s’est porté sur le transport Streamable HTTP plutôt que sur Stdio. Cette décision rend le serveur MCP découplé de son client, hautement disponible et évolutif. Elle garantit que le serveur pourra être interrogé par d’autres interfaces, services ou agents d’intelligence artificielle sans nécessiter de refonte de l’infrastructure.

Indépendamment du mécanisme de transport retenu, le format des échanges entre le client et le serveur est régi par la spécification JSON-RPC 2.0. Il s’agit d’un protocole d’appel de procédure à distance léger et sans état, qui structure les messages au format JSON (JSON-RPC Working Group 2013) [8].

Enfin, le serveur MCP de PadelContext adopte une architecture passerelle, telle qu’introduite dans l’état de l’art. Plutôt que d’accéder directement à la base de données, le serveur encapsule l’API REST existante du projet, sur laquelle s’appuient ses outils pour fournir les données et exécuter les actions

demandées par le modèle de langage. Ce choix permet de réutiliser la logique métier déjà implémentée dans l'API, de conserver une source de vérité unique et de maintenir une séparation claire des responsabilités entre les services.

7.5 Client MCP et modèle de langage

Le client web, qui joue le rôle de client MCP, constitue le point d'entrée du joueur dans PadelContext. Il héberge à la fois l'interface utilisateur et la logique d'orchestration entre le modèle de langage et le serveur MCP. Sa conception repose sur trois choix technologiques principaux, à savoir le framework, la bibliothèque d'intégration de l'intelligence artificielle et le modèle de langage utilisé.

Le choix s'est porté sur le framework Next.js, associé à React, afin de disposer d'un environnement full-stack unifié. Un projet de ce type impose en effet d'exécuter le client MCP et les appels au modèle de langage côté serveur, pour protéger les clés d'API ainsi que le secret d'accès au serveur MCP, tout en offrant au joueur une interface réactive capable de diffuser les réponses du modèle de langage. Next.js répond à ce besoin en réunissant, au sein d'une même application, le rendu de l'interface et l'exécution de code côté serveur.

À ce framework s'ajoute le Vercel AI SDK, une bibliothèque dédiée à l'intégration de modèles de langage dans les applications JavaScript. Elle fournit une abstraction commune pour dialoguer avec différents fournisseurs de modèles, un client MCP intégré, ainsi que la gestion native de la diffusion en continu des réponses et de l'orchestration des outils. Ce socle clé en main a permis de concentrer l'effort de développement sur la logique propre à PadelContext plutôt que sur la plomberie réseau entre le modèle, le serveur MCP et l'interface.

Concernant le modèle de langage, le choix s'est porté sur Google Gemini et plus précisément sur le modèle de langage `gemini-3.1-flash-lite`. Ce fournisseur a été retenu avant tout pour des raisons économiques, son offre proposant un palier gratuit généreux et un coût au token parmi les plus bas du marché, ce qui correspond au budget contraint d'un travail de bachelor. La variante "flash-lite" prolonge cette logique en privilégiant une faible latence et une consommation réduite, un atout pour un assistant sollicité de façon répétée. Le modèle prend par ailleurs en charge nativement l'appel d'outils,

capacité indispensable au fonctionnement du MCP.

Le développement de l'interface utilisateur du client a été confié à un agent de génération de code, selon une approche dite de "vibe coding". Ce parti pris se justifie par l'objet même du travail, qui porte sur la compréhension et la mise en œuvre du protocole MCP, et non sur l'esthétique ou l'ergonomie de l'interface. Le modèle utilisé pour la génération de l'interface est Claude Opus 4.8 d'Anthropic, dont les capacités sont largement suffisantes pour cette tâche.

7.6 Conteneurisation et gestion des environnements

Dans le cadre de ce projet, la conteneurisation de l'intégralité de l'écosystème applicatif s'est imposée comme une nécessité technique, motivée par des enjeux de reproductibilité et de facilité de déploiement. Pour répondre à ce besoin, j'ai choisi d'utiliser la plateforme de conteneurisation Docker.

Le fonctionnement de Docker repose sur l'utilisation d'images. Une image Docker est un modèle qui intègre toutes les dépendances nécessaires au fonctionnement d'une application et qui sert à créer des conteneurs. Un conteneur est alors une instance exécutable de cette image, qui fait tourner l'application dans un environnement isolé dont le résultat est identique quelle que soit l'infrastructure sous-jacente (trad. de Koneru 2025, p. 725) [9].

Cette isolation et cette reproductibilité générale garanties par Docker ont été les principaux arguments de son adoption pour PadelContext. La capacité de développer, tester et déployer l'application sur différents appareils sans se soucier du système d'exploitation hôte ou de ses configurations spécifiques s'est avérée extrêmement pratique.

De plus, afin d'optimiser les ressources, chaque conteneur de l'écosystème repose sur une image de base minimaliste de type Alpine Linux. Ces images allégées ne mettent à disposition que le strict nécessaire au fonctionnement de l'application, ce qui réduit considérablement la surface d'attaque potentielle et réduit le temps de déploiement des conteneurs.

Afin de prévenir l'obsolescence prématurée du projet et de minimiser les problèmes de compatibilité durant la phase de développement, un choix strict

de sélection des versions a été appliqué pour chaque image Docker. Le choix s'est systématiquement porté sur des versions actives, stables et officiellement maintenues, garantissant ainsi l'accès à une documentation à jour en cas de difficulté de codage.

Pour les images de l'API REST, du serveur MCP et du client MCP reposant tous sur l'environnement d'exécution JavaScript Node.js, l'image Docker `node:24.14.0-alpine3.23` a été sélectionnée. Ce choix est justifié par les recommandations officielles de Node.js, qui spécifient que la version 24 est une version *LTS (Long-Term Support)* durant la période de réalisation de ce travail de Bachelor. Ce statut LTS garantit la correction des failles de sécurité et des bugs critiques pendant une durée de 30 mois (Node.js 2026) [12].

Enfin, pour la base de données PostgreSQL, l'image `postgres:18.3-alpine3.23` a été retenue. Ce choix suit la même logique de pérennité en s'appuyant sur les recommandations officielles de PostgreSQL, qui identifient la version 18 comme la version majeure la plus récente et supportée à ce jour (PostgreSQL 2026) [13].

7.7 Modélisation de la base de données

La structure de la base de données relationnelle de PadelContext est présentée en deux temps. Le schéma relationnel donne une vue d'ensemble des entités et de leurs relations, tandis que le dictionnaire de données en détaille, table par table, les attributs, leurs types et leurs contraintes.

7.7.1 Schéma relationnel

Le schéma relationnel de la figure 8 représente les entités de l'application et les relations qui les lient, telles qu'elles sont générées par l'ORM Prisma à partir du modèle de données.

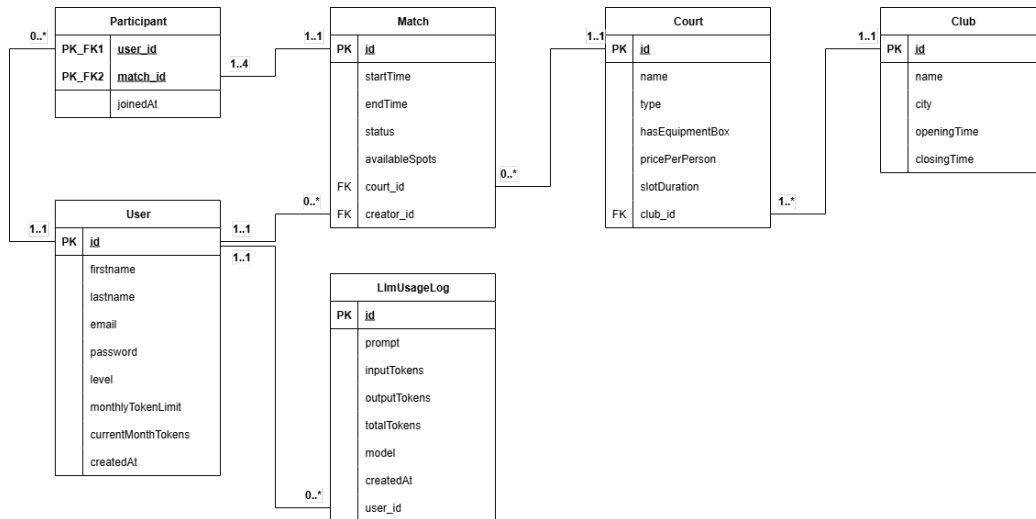


FIGURE 8 – Schéma relationnel de la base de données de l'application PadelContext

7.7.2 Dictionnaire de données

Le dictionnaire de données ci-dessous détaille la structure des tables de la base de données relationnelle (PostgreSQL) générée par l'ORM Prisma. Il est important de noter la présence de deux énumérations (*Enums*) utilisées pour restreindre les valeurs de certains champs :

- **CourtType** : Définit le type de terrain (INDOOR, OUTDOOR, COVERED).
- **MatchStatus** : Définit l'état actuel d'une partie (OPEN, COMPLETED, CANCELED).

User			
Attribut	Type	Contraintes / Clés	Description
id	Int	PK, Auto-incrément	Identifiant unique du joueur.
firstname	String	Requis	Prénom du joueur.
lastname	String	Requis	Nom de famille du joueur.
email	String	Requis, Unique	Adresse e-mail.
password	String	Requis	Mot de passe haché.
level	Int	Défaut : 1	Niveau du joueur.
createdAt	DateTime	Défaut : now()	Date et heure d'inscription sur la plateforme.
monthlyTokenLimit	Int	Défaut : 500000	Plafond mensuel de tokens autorisés pour l'usage de l'assistant conversationnel.
currentMonthTokens	Int	Défaut : 0	Tokens consommés sur le mois en cours.

TABLE 4 – Dictionnaire de données - Table User

Club			
Attribut	Type	Contraintes / Clés	Description
id	Int	PK, Auto-incrément	Identifiant unique du club.
name	String	Requis	Nom du club.
city	String	Requis	Ville où se situe le club.
postalCode	String	Requis	Code postal de l'emplacement du club.
openingTime	String	Requis	Heure d'ouverture habituelle quotidienne.
closingTime	String	Requis	Heure de fermeture habituelle quotidienne.

TABLE 5 – Dictionnaire de données - Table Club

Court			
Attribut	Type	Contraintes / Clés	Description
id	Int	PK, Auto-incrément	Identifiant unique du terrain.
name	String	Requis	Nom d'identification du terrain.
type	CourtType	Défaut : OUTDOOR	Type du terrain.
hasEquipmentBox	Boolean	Requis	Indique si le terrain met du matériel à disposition.
pricePerPerson	Float	Requis	Prix de la réservation par joueur.
slotDuration	Int	Requis	Durée standard d'un créneau de jeu en minutes.
club_id	Int	FK	Référence au club propriétaire du terrain.

TABLE 6 – Dictionnaire de données - Table Court

Match			
Attribut	Type	Contraintes / Clés	Description
id	Int	PK, Auto-incrément	Identifiant unique de la partie.
startTime	DateTime	Requis	Date et heure précises du début du match.
endTime	DateTime	Requis	Date et heure précises de la fin du match.
status	MatchStatus	Défaut : OPEN	État actuel du match.
availableSpots	Int	Requis	Nombre de places encore disponibles pour ce match.
court_id	Int	FK	Référence au terrain sur lequel se joue le match.
creator_id	Int	FK	Référence au joueur ayant initié la création du match.

TABLE 7 – Dictionnaire de données - Table Match

Participant			
Attribut	Type	Contraintes / Clés	Description
user_id	Int	PK, FK	Identifiant du joueur rejoignant le match.
match_id	Int	PK, FK	Identifiant du match rejoint.
joinedAt	DateTime	Défaut : now()	Date et heure de l'inscription du joueur à la partie.

TABLE 8 – Dictionnaire de données - Table Participant

LlmUsageLog			
Attribut	Type	Contraintes / Clés	Description
id	Int	PK, Auto-incrément	Identifiant unique de l'enregistrement.
user_id	Int	FK	Référence au joueur à l'origine de la requête.
createdAt	DateTime	Défaut : now()	Date et heure de la requête.
prompt	String	Requis	Texte de la requête envoyée au modèle de langage.
inputTokens	Int	Requis	Nombre de tokens en entrée.
outputTokens	Int	Requis	Nombre de tokens en sortie.
totalTokens	Int	Requis	Total des tokens consommés par la requête.
model	String	Requis	Identifiant du modèle de langage utilisé.

TABLE 9 – Dictionnaire de données - Table LlmUsageLog

7.8 Identification des sources de données météorologiques

Afin d'utiliser des données météorologiques réelles et concrètes, une phase de recherche a permis d'identifier MeteoSwiss comme source de qualité. Depuis mai 2025, cet Office fédéral de météorologie et de climatologie met progressivement à disposition ses données sous forme d'Open Government Data (OGD). Les données publiques ouvertes permettent, par exemple, aux villes d'optimiser en temps réel leurs transports publics, aux startups de développer des applications qui facilitent notre quotidien, et aux citoyens de suivre plus facilement les actions de leurs responsables politiques (Office fédéral de la statistique, 2026) [2].

L'accès aux données de MeteoSwiss s'effectue via une API REST conforme à la norme internationale OGC STAC. La famille de spécifications STAC (SpatioTemporal Asset Catalog) vise à standardiser la structuration et l'interrogation des métadonnées des ressources géospatiales. Les spécifications STAC définissent des types d'objets JSON liés par des relations de type lien afin de prendre en charge une interface navigable et une API RESTful offrant des interfaces de navigation et de recherche supplémentaires (trad. de STAC Community, 2025) [17].

Parmi l'ensemble des collections disponibles, la collection des données de prévisions locales a été sélectionnée. Celle-ci fournit des prévisions par point pour environ 6 000 sites en Suisse pour les 216 prochaines heures, mises à jour toutes les heures. Ces prévisions sont indispensables pour informer l'utilisateur des conditions météorologiques et donc de la faisabilité d'une partie de padel sur un terrain extérieur. Parmi tous les types de données météorologiques disponibles, trois types spécifiques ont été retenus, comme détaillé dans le tableau 10.

Identifiant	Groupe	Description
tre200h0	Température	Température de l'air à 2 m au-dessus du sol.
rp0003i0	Précipitation	Probabilité de précipitations pendant 3 heures.
fu3010h0	Vent	Valeur scalaire de la vitesse du vent, moyenne horaire en km/h.

TABLE 10 – Paramètres météorologiques MeteoSwiss choisis pour le projet PadelContext (MeteoSwiss, 2026) [10]

Les données de chaque paramètre sont distribuées sous forme de fichiers CSV. Chaque heure, un nouveau fichier est généré, contenant les prévisions des 216 prochaines heures pour l'ensemble du territoire suisse. La structure d'un fichier de données, par exemple pour la température (**tre200h0**), est organisée selon les en-têtes présentés dans le tableau 11.

Colonne CSV	Description
point_id	Identifiant unique de la station ou de la localité
point_type_id	Type de point (1 = Station, 2 = Code postal, 3 = Lieu d'intérêt).
Date	Horodatage du relevé au format : AAAAMMJJHHmm
tre200h0	Valeur de la température pour le créneau donné

TABLE 11 – Structure type des fichiers CSV de données de prévision locales de MeteoSwiss

Afin de lier une réservation de padel à une donnée météo précise, il est nécessaire de connaître le **point_id** correspondant à la localité du club. Pour ce faire, un fichier de référence fourni par MeteoSwiss recense l'intégralité des stations météorologiques, des codes postaux et des lieux d'intérêt. Ce fichier permet d'effectuer une jointure logique. Par exemple, le code postal 1226 (Thônex) est associé au **point_id** 122600.

8 Réalisation

8.1 API REST

8.1.1 Points de terminaison

Afin de garantir une maintenance aisée et une compréhension rapide de l'API, l'ensemble des points de terminaison a été documenté à l'aide de l'outil Swagger.

L'avantage de l'outil Swagger est qu'il permet de rédiger une documentation dynamique et accessible directement depuis le serveur de l'API. En effet, la documentation est générée automatiquement à partir de commentaires structurés directement intégrés au-dessus du code de chaque route de l'application. Cette méthode permet de centraliser le code et sa documentation au même endroit.

L'API PadelContext expose neuf points de terminaison couvrant l'authentification, la consultation et la création de matchs et de créneaux, l'accès aux données météorologiques ainsi que le suivi de la consommation du modèle de langage. Les tableaux 12 et 13 présentent une vue d'ensemble de ces routes.

Méthode	Endpoint	Description
POST	/api/auth/login	Authentifie un utilisateur via ses identifiants et retourne un JSON Web Token (JWT) valide pour authentifier ses futures requêtes.
GET	/api/matches	Récupère la liste des matchs ouverts (en attente de joueurs) en appliquant divers filtres optionnels (ville, prix, durée, etc.).
GET	/api/matches/me	Récupère tous les matchs de l'utilisateur authentifié, quel que soit leur statut, triés par date de début.
POST	/api/matches/from-slot	Convertit un créneau libre en un nouveau match ouvert et y inscrit automatiquement le créateur grâce à son JWT.
POST	/api/matches/{matchId}/join	Inscrit l'utilisateur authentifié à une partie existante disposant encore de places disponibles grâce à son JWT.
GET	/api/available-slots	Retourne les créneaux horaires disponibles (les 7 prochains jours par défaut, ou une fenêtre précisée) en appliquant divers filtres optionnels (ville, prix, durée, etc.).

TABLE 12 – Points de terminaison de l'API - Partie 1

Méthode	Endpoint	Description
GET	<code>/api/weather</code>	Fournit les données météorologiques (précipitations, vent, température) pour un code postal suisse et une date donnée.
GET	<code>/api/llm-usage/me</code>	Retourne le nombre de tokens consommés sur le mois en cours ainsi que la limite mensuelle de l'utilisateur.
POST	<code>/api/llm-usage/log</code>	Enregistre un usage du modèle de langage et incrémente le compteur mensuel de tokens de l'utilisateur.

TABLE 13 – Points de terminaison de l'API - Partie 2

La spécification technique complète et exhaustive de ces points de terminaison, incluant les paramètres de requête, le format des corps JSON attendus, ainsi que la gestion des différents codes de statut HTTP, est visualisable via l'interface graphique de Swagger.

Des captures d'écran détaillant l'utilisation et les réponses de chaque point de terminaison via cette interface sont disponibles dans l'annexe 11.2.

8.1.2 Acquisition des données météorologiques

Comme présenté lors de la conception, l'accès aux données météorologiques constitue un élément central de la solution PadelContext. Initialement envisagé comme un outil MCP interrogé directement par le modèle de langage lors de la planification du projet, il a finalement été réalisé sous la forme d'un service interne à l'API REST. Ce choix centralise la logique d'acquisition et de mise en cache des prévisions au sein de l'API. Le serveur MCP se contente alors d'appeler le point de terminaison `/api/weather` lorsqu'un terrain extérieur est concerné. Cette décision répond également à un objectif d'optimisation du coût en tokens du modèle de langage, détaillé dans l'évaluation des résultats.

Cette acquisition repose sur un service dédié, conçu sous la forme d'un singleton. Une instance unique est ainsi partagée dans toute l'application, ce qui

garantit que les données téléchargées en arrière-plan et celles lues lors d'une requête proviennent du même cache en mémoire. Au démarrage, le service charge en mémoire un fichier CSV statique fourni par MeteoSwiss qui associe chaque code postal suisse à un identifiant de point de prévision (`point_id`). Les prévisions sont ensuite conservées dans un cache en mémoire vive, organisé par catégorie de mesure. Trois catégories sont retenues, la température, la probabilité de précipitations et la vitesse du vent.

La mise à jour de ce cache est confiée à une tâche planifiée qui s'exécute toutes les dix minutes, complétée par une première exécution immédiate au démarrage afin de ne jamais servir un cache vide. À chaque exécution, la tâche interroge l'API STAC de MeteoSwiss pour détecter la publication de nouveaux fichiers de prévision. Deux mécanismes limitent le coût de cette synchronisation. D'une part, l'en-tête HTTP `ETag` permet de ne re-télécharger un fichier que s'il a réellement changé, une réponse HTTP 304 signalant l'absence de modification. D'autre part, un contrôle d'intégrité par somme de contrôle SHA-256 vérifie que chaque fichier téléchargé n'a pas été corrompu avant de remplacer les données du cache.

Grâce à cette architecture, le point de terminaison `/api/weather` répond instantanément. Lorsqu'une requête arrive, le service convertit le code postal en identifiant de point puis lit directement la valeur correspondante dans le cache en mémoire, sans solliciter l'API MeteoSwiss. Cette mise en cache en amont garantit des temps de réponse constants et découple la disponibilité de l'application de celle de l'API externe.

8.1.3 Stratégie de tests

Afin de garantir la fiabilité, la robustesse et la non-régression de l'API REST tout au long de son développement, une stratégie de test rigoureuse a été mise en place. En accord avec les choix technologiques définis précédemment, le framework Jest a été utilisé pour orchestrer ces vérifications. La stratégie adoptée repose sur deux niveaux de validation complémentaires, les tests unitaires et les tests d'intégration.

Les tests unitaires ont pour objectif de valider le comportement d'un composant de code de manière totalement isolée du reste du système. Dans le cadre de l'API PadelContext, ces tests se concentrent exclusivement sur la

logique métier des contrôleurs. Pour isoler efficacement chaque contrôleur, toutes les dépendances externes ont été simulées. Par exemple, les interactions avec la base de données via Prisma, le hachage des mots de passe avec la bibliothèque `bcryptjs` ou encore la génération de tokens avec la bibliothèque `jsonwebtoken` ont été remplacés par de fausses implémentations contrôlées par Jest. L'intérêt principal de cette approche est sa rapidité d'exécution et sa grande précision. En simulant des erreurs liées à la base de données, comme l'échec d'une transaction ou une perte de connexion, il a été possible de s'assurer que l'API gère correctement les exceptions et renvoie les bons codes HTTP sans avoir besoin d'altérer un environnement réel.

Contrairement aux tests unitaires, les tests d'intégration visent à vérifier que l'ensemble des composants du système communiquent correctement entre eux. Ils valident le cheminement complet d'une requête HTTP, du routeur au middleware permettant la vérification du JWT, en passant par le contrôleur, jusqu'à l'écriture ou la lecture dans une véritable base de données PostgreSQL. Ces tests ont été réalisés à l'aide de la bibliothèque Supertest, qui permet de simuler des requêtes HTTP réelles vers l'application Express. Lors de ces validations, des jeux de données réels sont créés en amont puis supprimés en aval afin de ne pas polluer l'environnement de développement. Bien qu'un contrôleur puisse parfaitement fonctionner de manière isolée, il peut échouer lorsqu'il est connecté à la base de données réelle en raison, par exemple, d'une contrainte de clé étrangère non respectée ou d'une erreur de format de date. Les tests d'intégration garantissent ainsi que des actions critiques, telles que la création d'un match depuis un créneau libre ou la participation à un match, fonctionnent de manière transactionnelle et fiable dans des conditions d'utilisation réelles.

8.2 Serveur MCP

8.2.1 Définitions des outils MCP

Le serveur Model Context Protocol (MCP) constitue le pont de communication essentiel entre l'Assistant IA et les sources de données du système. Son rôle est d'exposer de manière standardisée les points de terminaison de l'API REST de PadelContext, ainsi que l'API externe de MeteoSwiss, sous forme d'outils (tools) directement exécutables par l'intelligence artificielle.

Afin de couvrir l'ensemble des cas d'usage définis lors de la modélisation fonctionnelle, le serveur MCP expose cinq outils distincts. Le tableau 14 résume leurs objectifs respectifs.

Nom de l'outil	Source interrogée	Description fonctionnelle
<code>get-matches</code>	API PadelContext	Récupère la liste des matchs disponibles via le point de terminaison correspondant.
<code>get-available-slots</code>	API PadelContext	Récupère la liste des créneaux horaires disponibles via le point de terminaison correspondant.
<code>create-match-from-slot</code>	API PadelContext	Crée un match à partir d'un créneau horaire libre et y inscrit l'utilisateur authentifié via le point de terminaison correspondant.
<code>join-match</code>	API PadelContext	Inscrit l'utilisateur authentifié à un match existant via le point de terminaison correspondant.
<code>get-meteo-swiss</code>	API MeteoSwiss	Fournit les prévisions climatiques locales pour valider la faisabilité d'une partie en extérieur.

TABLE 14 – Liste des outils exposés par le serveur MCP de PadelContext

Dans l'architecture du Model Context Protocol, la description sémantique

d'une fonctionnalité est critique. Pour comprendre l'objectif et les spécificités d'un outil, les LLMs s'appuient exclusivement sur sa description en langage naturel. Une étude récente, ayant analysé 856 outils MCP, révèle que 97,1 % d'entre eux contiennent des défauts de conception qualifiés de « Tool Smells », avec 56 % des outils échouant même à définir clairement leur objectif principal (Hasan et al. 2026, p. 4) [7]. Les auteurs soulignent que si ces descriptions sont défectueuses, sous-spécifiées ou trompeuses, l'agent IA risque de sélectionner le mauvais outil ou de fournir des arguments invalides, réduisant considérablement la fiabilité du système.

Bien que l'étude démontre que l'amélioration de ces descriptions permet d'augmenter le taux de succès des tâches, elle met également en garde sur le fait qu'une documentation trop volumineuse augmente le coût et le nombre d'étapes d'exécution. Il est donc nécessaire de trouver un juste équilibre (Hasan et al. 2026, p. 4) [7].

Pour PadelContext, les outils ont été rigoureusement structurés en s'appuyant sur la grille d'évaluation d'excellence (score de 5/5) proposée par l'étude mentionnée. Chaque description est structurée autour de six piliers sémantiques afin d'atteindre une qualité optimale (Hasan et al. 2026, p. 42) [7] :

1. **Purpose** : Explique clairement la fonction, le comportement et les données de retour de l'outil en utilisant un langage précis.
2. **Usage Guidelines** : Énonce explicitement les cas d'utilisation appropriés et les cas où l'outil ne doit pas être utilisé, incluant une désambiguïsation si le nom de l'outil s'avère ambigu.
3. **Limitation** : Indique clairement ce que l'outil ne retourne pas, les limites de sa portée ainsi que ses contraintes importantes.
4. **Parameter Explanation** : Chaque paramètre est détaillé avec son type, sa signification, son effet sur le comportement de l'outil, et son statut.
5. **Examples vs. Description Balance** : La description textuelle se suffit à elle-même, les exemples fournis servent uniquement à compléter l'explication et non à la remplacer.
6. **Length and Completeness** : La description de l'outil est composée de quatre phrases ou plus, rédigées dans un style concis et bien structuré, couvrant tous les aspects de son utilisation.

L'application stricte de cette grille garantit une réduction drastique des erreurs d'interprétation par l'agent IA, assurant ainsi une orchestration fiable des requêtes tout en optimisant la pertinence des résultats.

8.3 Client MCP et assistant IA

8.4 Fiabilité de l'assistant

8.5 Maîtrise des coûts

8.6 Cas d'usage

Exemple d'une réservation d'un terrain de padel en extérieur, avec un match-making automatique et une vérification de la météo en temps réel

9 Évaluation des résultats

9.1 Bénéfice UX

9.2 Fiabilité face aux hallucinations

9.3 Viabilité des coûts et complexité

10 Conclusion

10.1 Conclusion générale

10.2 Améliorations possibles

Références

- [1] PlayPad | Consultez notre FAQ et trouvez vos réponses. URL : <https://www.playpadapp.com/faqs>.
- [2] Qu'est-ce que l'Open Government Data (OGD)? - Flyer - | Publication, February 2026. URL : <https://www.bfs.admin.ch/asset/fr/36463590>.

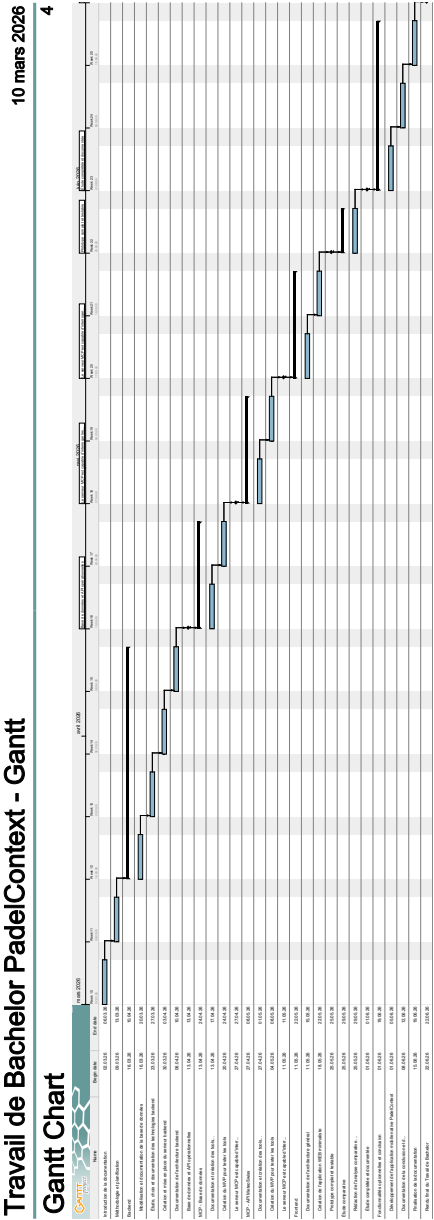
- [3] Sireesha Addanki. Architectural Shifts in API Design : The Progression from SOAP to REST and GraphQL. *IJSAT - International Journal on Science and Technology*, 16(2), June 2025. URL : <https://www.ijsat.org/research-paper.php?id=6579>, doi: 10.71097/IJSAT.v16.i2.6579.
- [4] Anthropic. Introducing the Model Context Protocol, November 2024. URL : <https://www.anthropic.com/news/model-context-protocol>.
- [5] Agile Business. MoSCoW Prioritisation - DSDM Project Framework Handbook | Agile Business Consortium. URL : <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>.
- [6] Anujkumarsinh Donvir. A Comparative Analysis of JavaScript Unit and End-to-End Testing Frameworks : Enhancing Quality Assurance in Modern Web Applications. In *2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)*, pages 1289–1296, December 2024. ISSN : 2472-7555. URL : <https://ieeexplore.ieee.org/document/10847365>, doi:10.1109/CICN63059.2024.10847365.
- [7] Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions, February 2026. arXiv :2602.14878 [cs]. URL : <http://arxiv.org/abs/2602.14878>, doi:10.48550/arXiv.2602.14878.
- [8] JSON-RPC Working Group. JSON-RPC 2.0 Specification, January 2013. URL : <https://www.jsonrpc.org/specification>.
- [9] Naga Murali Krishna Koneru. Containerization Best Practices- Using Docker and Kubernetes for Enterprise Applications. *Journal of Information Systems Engineering and Management*, 10(45s) :724–746, April 2025. URL : <https://jisem-journal.com/index.php/journal/article/view/8905>, doi:10.52783/jisem.v10i45s.8905.
- [10] MeteoSwiss. Local forecast data | Open Data Documentation, 2026. URL : <https://opendatadocs.meteoswiss.ch/e-forecast-data/e4-local-forecast-data#data-structure>.

- [11] Model Context Protocol. Transports, November 2025. URL : <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>.
- [12] Node.js. Node.js — Node.js Releases, May 2026. URL : <https://nodejs.org/en/about/previous-releases>.
- [13] PostgreSQL. PostgreSQL : Versioning Policy, 2026. URL : <https://www.postgresql.org/support/versioning/>.
- [14] Kosovare Sahatqija, Jaumin Ajdari, Xhemal Zenuni, Bujar Raufi, and Florije Ismaili. Comparison between relational and NOSQL databases. In *2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pages 0216–0221, May 2018. URL : <https://ieeexplore.ieee.org/document/8400041>, doi:10.23919/MIPRO.2018.8400041.
- [15] Sanket Vilas Salunke and Abdelkader Ouda. A Performance Benchmark for the PostgreSQL and MySQL Databases. *Future Internet*, 16(10) :382, October 2024. URL : <https://www.mdpi.com/1999-5903/16/10/382>, doi:10.3390/fi16100382.
- [16] Luis Miguel Vieira da Silva, Aljosha Köcher, and Felix Gehlhoff. Beyond Formal Semantics for Capabilities and Skills : Model Context Protocol in Manufacturing. In *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–4, September 2025. arXiv :2506.11180 [cs.SE]. URL : <http://arxiv.org/abs/2506.11180>, doi:10.1109/ETFA65518.2025.11205601.
- [17] STAC Community. SpatioTemporal Asset Catalog (STAC) Community Standard, 2025. URL : <https://docs.ogc.org/cs/25-004/25-004.html>.
- [18] Stack Overflow. Technology | 2025 Stack Overflow Developer Survey, 2025. URL : <https://survey.stackoverflow.co/2025/technology/>.
- [19] Suchetha Vijayakumar, Krishna Prasad K, and Raviraja Holla M. Assessing the Effectiveness of MoSCoW Prioritization in Software Development : A Holistic Analysis across Methodologies. *EAI Endorsed Transactions on Internet of Things*, 10, October 2024. URL : <https://publications.eai.eu/index.php/IoT/article/view/6515>, doi:10.4108/eetiot.6515.

- [20] Tom Dohnal. Build A Remote MCP Server With TypeScript (Streamable HTTP), July 2025. URL : <https://www.youtube.com/watch?v=nTPeM813zTg>.
- [21] Serhii Zhadko-Bazilevych. Analysis of ORM framework approaches for Node.js. *Journal of Computer Sciences Institute*, 37 :426–430, December 2025. URL : <https://ph.pollub.pl/index.php/jcsi/article/view/7951>, doi:10.35784/jcsi.7951.

11 Annexes

11.1 Diagramme gantt du projet PadelContext



11.2 Documentation Swagger de l'API PadelContext

28/04/2026 22:26

Swagger UI

Padel Context API

1.0.0 OAS 3.0

API du travail de bachelor Padel Context

Authorize

Authentication

POST /api/auth/login Authentifier un utilisateur

Permet à un utilisateur de s'authentifier en fournissant son email et son mot de passe. Retourne un token JWT et les informations de l'utilisateur.

Parameters

Try it out

No parameters

Request body required

application/json

Les identifiants de l'utilisateur

Example Value

Schema

```
{  "email": "jonathan.borel@padelcontext.com",  "password": "pomme123"}
```

Responses

localhost:3000/api-docs/#/Matches/post_api_matches__matchId__join

1/13

28/04/2026 22:26

Swagger UI

Code	Description	Links
200	Authentification réussie	No links
	Media type application/json Controls Accept header. Example Value Schema <pre>{ "message": "login successful", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "id": "123e4567-e89b-12d3-a456-426614174000", "firstname": "John", "lastname": "Doe", "email": "jonathan.borel@padelcontext.com", "level": "intermédiaire" }}</pre>	
400	Erreur de validation - Email ou mot de passe manquants ou invalides	No links
	Media type application/json Examples Paramètres manquants Example Value Schema <pre>{ "message": "email and password are required"}</pre>	
401	Authentification échouée - Email non trouvé ou mot de passe incorrect	No links
	Media type application/json Examples Identifiants invalides Example Value Schema <pre>{ "message": "invalid credentials"}</pre>	
500	Erreur serveur interne	No links
	Media type application/json Example Value Schema <pre>{ "message": "Error while logging in"}</pre>	

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

2/13

28/04/2026 22:26

Swagger UI

Code	Description	Links
------	-------------	-------

Available Slots

GET

/api/available-slots

Récupérer les créneaux disponibles à l'aide de filtres

Retourne les créneaux disponibles pour chaque terrain correspondant aux filtres.

Sans l'utilisation de `timeFrom` et `timeTo` , retourne les créneaux pour les 7 prochains jours.

Les créneaux déjà occupés par des matchs ouverts et complétés sont exclus. Les matchs annulés ne bloquent pas un créneau.

Les filtres query sont optionnels et combinables.

Parameters

Try it out

Name	Description
city string (query)	Ville du club. Example : Lancy Lancy
courtType string (query)	Type de terrain. La valeur est normalisée en majuscules. Une valeur hors enum est ignorée. Available values : INDOOR, OUTDOOR, COVERED Example : COVERED COVERED
hasEquipmentBox boolean (query)	Filtrer selon la présence d'une box d'équipement. Example : true true
minPricePerPerson number (query)	Prix minimum par personne (inclus). Example : 10 10
maxPricePerPerson number	Prix maximum par personne (inclus).

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

3/13

28/04/2026 22:26

Swagger UI

Name	Description
<i>(query)</i>	<i>Example : 25</i>
	25
slotDuration integer <i>(query)</i>	Durée exacte du créneau en minutes. <i>Example : 60</i>
	60
minSlotDuration integer <i>(query)</i>	Durée minimale du créneau en minutes (incluse). <i>Example : 45</i>
	45
maxSlotDuration integer <i>(query)</i>	Durée maximale du créneau en minutes (incluse). <i>Example : 120</i>
	120
timeFrom string(\$date-time) <i>(query)</i>	Début de la fenêtre de disponibilité à retourner. <i>Example : 2026-04-15T08:00:00.000Z</i>
	2026-04-15T08:00:00.000Z
timeTo string(\$date-time) <i>(query)</i>	Fin de la fenêtre de disponibilité à retourner. <i>Example : 2026-04-15T20:00:00.000Z</i>
	2026-04-15T20:00:00.000Z

Responses

Code	Description	Links
200	Liste des terrains avec leurs créneaux disponibles.	No links

Media type

Controls Accept header.

Example Value

Schema

Examples

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

4/13

28/04/2026 22:26

Swagger UI

Code	Description	Links
	<pre>[{ "court": { "id": 11, "name": "Court Alpha", "type": "INDOOR", "hasEquipmentBox": true, "pricePerPerson": 18, "slotDuration": 60, "club": { "name": "Geneva Club", "city": "Lancy", "openingTime": "08:00", "closingTime": "10:00" } }, "availableSlots": [{ "startTime": "2026-04-15T09:00:00.000Z", "endTime": "2026-04-15T10:00:00.000Z" }] }]</pre>	
400	Fenêtre invalide (<code>timeTo</code> <= <code>timeFrom</code>).	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "message": "timeTo must be greater than timeFrom" }</pre>	
500	Erreur interne lors de la récupération des créneaux disponibles.	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "message": "Error while fetching available slots" }</pre>	

Matches

GET

/api/matches

Récupérer les matchs ouverts à l'aide de filtres

Retourne uniquement les matchs ouverts à venir, soit un match avec au moins une place disponible.

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

5/13

28/04/2026 22:26

Swagger UI

Les filtres query sont optionnels et combinables.

Parameters

Try it out

Name	Description
city string (query)	Ville du club. <i>Example</i> : Lancy Lancy
courtType string (query)	Type de terrain. La valeur est normalisée en majuscules. Une valeur hors enum est ignorée. <i>Available values</i> : INDOOR, OUTDOOR, COVERED <i>Example</i> : INDOOR INDOOR
hasEquipmentBox boolean (query)	Filtrer selon la présence d'une box d'équipement. <i>Example</i> : true true
minPricePerPerson number (query)	Prix minimum par personne (inclus). <i>Example</i> : 10 10
maxPricePerPerson number (query)	Prix maximum par personne (inclus). <i>Example</i> : 20 20
slotDuration integer (query)	Durée exacte du créneau en minutes. <i>Example</i> : 120 120
minSlotDuration integer (query)	Durée minimale du créneau en minutes (incluse). <i>Example</i> : 90 90

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

6/13

28/04/2026 22:26

Swagger UI

Name	Description
maxSlotDuration integer (query)	Durée maximale du créneau en minutes (incluse). Example : 120 120
availableSpots integer (query)	Nombre exact de places disponibles. Example : 2 2
minAvailableSpots integer (query)	Nombre minimal de places disponibles (inclus). Example : 1 1
startTimeFrom string(\$date-time) (query)	Date/heure minimale de début. Si la valeur est dans le passé, la borne utilisée reste la date actuelle. Example : 2026-04-15T18:00:00.000Z 2026-04-15T18:00:00.000Z
startTimeTo string(\$date-time) (query)	Date/heure maximale de début (incluse). Example : 2026-04-16T20:00:00.000Z 2026-04-16T20:00:00.000Z
endTimeFrom string(\$date-time) (query)	Date/heure minimale de fin (incluse). Example : 2026-04-15T19:00:00.000Z 2026-04-15T19:00:00.000Z
endTimeTo string(\$date-time) (query)	Date/heure maximale de fin (incluse). Example : 2026-04-15T21:00:00.000Z 2026-04-15T21:00:00.000Z
participantAverageLevel number (query)	Niveau moyen cible des participants déjà inscrits. Example : 5 5

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

7/13

28/04/2026 22:26

Swagger UI

Name	Description
participantAverageLevelTolerance	5 Tolérance absolue autour de participantAverageLevel . number (query) Default value : 0.5 Example : 0.1 0.1

Responses

Code	Description	Links
200	Liste des matchs correspondant aux filtres. Media type application/json Controls Accept header. Example Value Schema [{ "id": 101, "startTime": "2026-04-15T18:00:00.000Z", "endTime": "2026-04-15T20:00:00.000Z", "status": "OPEN", "availableSpots": 2, "count": { "name": "Court Alpha", "type": "INDOOR", "hasEquipmentBox": true, "pricePerPerson": 18, "slotDuration": 120, "club": { "name": "Geneva Club", "city": "Lancy", "openingTime": "08:00", "closingTime": "23:00" } } }, "participants": [{ "user": { "firstname": "Jonathan", "lastname": "Borel-Jaquet", "email": "jonathan.borel@padelcontext.com", "level": 2 } }]]	No links
500	Erreur interne lors de la récupération des matchs. Media type	No links

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

8/13

28/04/2026 22:26

Swagger UI

Code	Description	Links
	application/json Example ValueSchema <pre>{ "message": "Error while fetching matches"}</pre>	

POST

/api/matches/from-slot

Cr  er un match depuis un cr  neau disponible

Permet    un utilisateur authentifi   de cr  er un match    partir d'un slot disponible, puis de s'y inscrire automatiquement.

Parameters

Try it out

No parameters

Request body required

application/json

Informations du cr  neau    transformer en match.
Example ValueSchema

```
{  "courtId": 12,  "startTime": "2026-04-15T18:00:00.000Z",  "endTime": "2026-04-15T19:00:00.000Z"}
```

Responses

Code	Description	Links
201	Match cr���� avec succ��s depuis le cr��neau. Media type application/json Controls Accept header. Example ValueSchema	No links

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

9/13

28/04/2026 22:26

Swagger UI

Code

```
"availableSpots": 3,
"startTime": "2026-04-15T18:00:00.000Z",
"endTime": "2026-04-15T19:00:00.000Z",
"creator_id": 12,
"court": {
  "name": "Court Alpha",
  "type": "INDOOR",
  "club": {
    "name": "Geneva Club",
    "city": "Lancy"
  }
},
"participants": [
  {
    "user": {
      "id": 21,
      "firstname": "Jonathan",
      "lastname": "Borel-Jaquet",
      "email": "jonathan.borel@padelcontext.com",
      "level": 2
    },
    "joinedAt": "2026-04-15T17:30:00.000Z"
  }
]
},
]
```

Links

400

Données invalides ou créneau non conforme aux règles métier.

No links

Media type

Examples

application/json

Paramètres requis manquants

Example Value

Schema

{
 "message": "courtId, startTime and endTime are required"
}

401

Utilisateur non authentifié.

No links

Media type

application/json

Example Value

Schema

{
 "message": "unauthorized"
}

404

Terrain introuvable.

No links

Media type

application/json

Example Value

Schema

{
 "message": "court not found"
}

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

10/13

28/04/2026 22:26

Swagger UI

Code	Description	Links
409	Le créneau n'est plus disponible. Media type application/json Example Value Schema <pre>{ "message": "slot is no longer available" }</pre>	No links
500	Erreur interne lors de la création du match depuis le créneau. Media type application/json Example Value Schema <pre>{ "message": "Error while creating match from slot" }</pre>	No links

POST

/api/matches/{matchId}/join

Rejoindre un match ouvert



Permet à un utilisateur authentifié de rejoindre un match ouvert s'il reste des places disponibles.

Parameters

Try it out

Name	Description
matchId ★ required integer (path)	Identifiant du match à rejoindre. Example : 101 101

Responses

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

11/13

28/04/2026 22:26

Swagger UI

Code	Description	Links
200	Match rejoint avec succès. Media type application/json Controls Accept header. Example ValueSchema <pre>{ "message": "successfully joined match", "match": { "id": 101, "status": "OPEN", "availableSpots": 1, "startTime": "2026-04-15T18:00:00.000Z", "endTime": "2026-04-15T20:00:00.000Z", "creator_id": 12, "court": { "name": "Court Alpha", "type": "INDOOR", "club": { "name": "Geneva Club", "city": "Lancy" } } }, "participants": [{ "user": { "id": 21, "firstname": "Jonathan", "lastname": "Borel-Jaquet", "email": "jonathan.borel@padelcontext.com", "level": 2 }, "joinedAt": "2026-04-15T17:30:00.000Z" }] }</pre>	No links
400	Identifiant de match invalide, match fermé, plus de place indisponible ou utilisateur déjà inscrit. Media typeExamples application/jsonIdentifiant invalide Example ValueSchema <pre>{ "message": "invalid match ID" }</pre>	No links
401	Utilisateur non authentifié. Media type application/json Example ValueSchema <pre>{ "message": "unauthorized" }</pre>	No links

localhost:3000/api-docs/#!/Matches/post_api_matches__matchId__join

12/13

28/04/2026 22:26Swagger UI

Code	Description	Links
404	Match introuvable. Media type application/json Example ValueSchema { "message": "match not found" }	No links
500	Erreur interne lors de la participation au match. Media type application/json Example ValueSchema { "message": "Error while joining match" }	No links